TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



TRƯƠNG MINH THÀNH - MSSV: 524H0032
TRƯƠNG CAO SANG - MSSV: 524H0125

XÂY DỰNG MÔ PHỎNG HỆ THỐNG CLOUD THU NHỎ (MINICLOUD)

BÁO CÁO CUỐI KỲ
ĐIỆN TOÁN ĐÁM MÂY

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



TRƯƠNG MINH THÀNH - MSSV: 524H0032

TRƯƠNG CAO SANG - MSSV: 524H0125

XÂY DỰNG MÔ PHỎNG HỆ THỐNG CLOUD THU NHỎ (MINICLOUD)

BÁO CÁO CUỐI KỲ

ĐIỆN TOÁN ĐÁM MÂY

Giảng viên hướng dẫn

TS. Bùi Quy Anh

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất đến TS. Bùi Quy Anh, người đã tận tâm truyền đạt kiến thức chuyên môn vững chắc, nền tảng tư duy khoa học và phương pháp luận nghiên cứu cho chúng em trong suốt quá trình học tập. Nhờ những bài giảng giàu tính thực tiễn, sự định hướng chuẩn mực và những tài liệu học tập chi tiết của Thầy, chúng em đã có cơ hội tiếp cận và hiện thực hóa thành công kiến trúc mô phỏng một nền tảng Cloud Platform hoàn chỉnh. Đề tài này không chỉ giúp chúng em củng cố lý thuyết về ảo hóa, containerization, mạng phân tán và bảo mật đám mây, mà còn rèn luyện cho chúng em tác phong làm việc nhóm kỹ thuật chuẩn mực và kỹ năng triển khai thực tế trên hạ tầng đám mây công cộng Amazon Web Services (AWS).

TP. Hồ Chí Minh, ngày 02 tháng 11 năm 2026

Tác giả 1



Trương Minh Thành

Tác giả 2

Trương Cao Sang

Trương Cao Sang

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH

TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng em xin cam đoan đề tài "Xây dựng mô phỏng hệ thống Cloud thu nhỏ (MiniCloud) trên nền tảng Container và điện toán đám mây AWS" là công trình nghiên cứu và thực hành thực tế do chính nhóm chúng em thực hiện dưới sự hướng dẫn khoa học của TS. Bùi Quy Anh. Toàn bộ mã nguồn, cấu hình Docker Compose, kịch bản triển khai trên AWS EC2 và số liệu kiểm thử được trình bày trong báo cáo này là hoàn toàn trung thực, chính xác và do các thành viên trong nhóm trực tiếp xây dựng, kiểm thử và ghi nhận.

Các tài liệu tham khảo, thư viện mã nguồn mở và nền tảng công nghệ được kế thừa trong đề tài đều được trích dẫn, chú thích rõ ràng nguồn gốc và tuân thủ tuyệt đối chuẩn mực liêm chính học thuật của Trường Đại học Tôn Đức Thắng. Chúng em xin chịu hoàn toàn trách nhiệm trước Nhà trường và Khoa nếu có bất kỳ sự thiếu trung thực nào xảy ra.

Nếu phát hiện có bất kỳ sự gian lận nào chúng em xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng em gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 02 tháng 11 năm 2026

Tác giả 1 *Tác giả 2*



Trương Cao Sang

Trương Minh Thành *Trương Cao Sang*

XÂY DỰNG MÔ PHỎNG HỆ THỐNG CLOUD THU NHỎ (MINICLOUD)

TÓM TẮT

Đề tài tập trung nghiên cứu, thiết kế và hiện thực hóa một hệ thống điện toán đám mây thu nhỏ mang tên MiniCloud, mô phỏng toàn diện các thành phần cốt lõi trong hạ tầng của các nhà cung cấp đám mây hàng đầu thế giới (như AWS, Azure, GCP). Hệ thống bao gồm 9 loại máy chủ chuyên biệt vận hành độc lập dưới dạng container Docker: Web Frontend Server, Application Backend Server, Relational Database Server (MariaDB), Authentication & Identity Server (Keycloak OIDC), Object Storage Server (MinIO S3), Internal DNS Server (BIND9), Monitoring Server (Prometheus), Logging/Visualization Server (Grafana), và API Gateway/Reverse Proxy (Nginx). Toàn bộ các máy chủ được liên kết thông qua mạng ảo hóa cô lập cloud-net (Subnet: 172.28.0.0/16) với dải địa chỉ IP tĩnh cố định.

Bên cạnh 9 máy chủ cơ bản đạt điểm chuẩn mực, nhóm đã phát triển thành công 10 tính năng mở rộng ứng dụng thực tế chuyên sâu: xây dựng blog cá nhân đa trang, phát triển API danh sách sinh viên đọc từ JSON và truy vấn trực tiếp từ MariaDB qua PyMySQL, thiết lập bảo mật định danh OpenID Connect (OIDC) với JWT Bearer Token, lưu trữ và cấu hình quyền tải công khai cho đối tượng trên MinIO S3, mở rộng bản ghi phân giải tên miền nội bộ BIND9, thu thập metrics hiệu năng từ web nodes qua Prometheus, trực quan hóa sức khỏe hệ thống với Grafana Custom Dashboard, định tuyến API Gateway tập trung, và cân bằng tải Round Robin luân phiên giữa 2 web frontend nodes. Hệ thống đã được đóng gói và phát hành 3 Docker Images lên Docker Hub (bevissava), đồng thời triển khai hoạt động ổn định trên máy ảo AWS EC2 (Ubuntu 22.04 LTS) với địa chỉ IP công khai, giải quyết triệt để các thách thức về bộ nhớ ảo (Swap RAM) và phân quyền lưu trữ.

BUILDING A SIMULATION OF MINICLOUD SYSTEM

ABSTRACT

This project focuses on the research, architectural design, and practical implementation of a micro-scale cloud computing platform named MiniCloud, comprehensively simulating the core infrastructure components of major Cloud Service Providers (CSPs) such as Amazon Web Services (AWS), Azure, and Google Cloud Platform (GCP). The system incorporates nine fundamental server roles running as decoupled Docker containers: Web Frontend Server, Application Backend Server, Relational Database Server (MariaDB), Authentication & Identity Server (Keycloak OIDC), Object Storage Server (MinIO S3), Internal DNS Server (BIND9), Monitoring Server (Prometheus), Logging/Visualization Server (Grafana), and an API Gateway / Load Balancer (Nginx). All containers communicate securely within an isolated virtual bridge network designated as cloud-net (Subnet: 172.28.0.0/16) utilizing static IP allocations.

In addition to the baseline requirements, the project successfully implements ten advanced, production-grade cloud features: a multi-page responsive personal blog, REST API endpoints querying structured JSON data and persistent relational databases via PyMySQL, centralized identity management based on OpenID Connect (OIDC) with RS256 JWT Bearer Token verification, object storage bucket policies for public asset retrieval, custom private DNS zone resolutions, Prometheus scrape configurations for web nodes, an automated Grafana System Health monitoring dashboard, unified reverse proxy path routing, and Round Robin load balancing across dual web frontend instances. The complete architecture was containerized, published to Docker Hub registry (bevissava), and successfully deployed on an AWS EC2 instance (Ubuntu 22.04 LTS), resolving real-world technical constraints regarding virtual memory (Swap RAM allocation) and container networking.

Mục lục

DANH MỤC HÌNH VẼ viii

DANH MỤC BẢNG BIỂU x

DANH MỤC CÁC CHỮ VIẾT TẮT xi

CHƯƠNG 1. GIỚI THIỆU VÀ MỤC TIÊU ĐỀ TÀI 1

1.1 Tính cấp thiết và bối cảnh đề tài 1

1.2 Mục tiêu nghiên cứu và phạm vi đề tài 1

1.3 Phân tích vai trò và yêu cầu 9 loại máy chủ trong hệ thống MiniCloud 2

1.4 Đối chiếu kiến trúc MiniCloud với hệ sinh thái dịch vụ trên Amazon Web Services (AWS) 3

CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC VÀ SƠ ĐỒ MẠNG HỆ THỐNG 5

2.1 Kiến trúc tổng thể hệ thống MiniCloud (Multi-tier Architecture) 5

2.2 Thiết kế kiến trúc mạng ảo hóa Docker Network (cloud-net: 172.28.0.0/16) 6

2.3 Luồng xử lý yêu cầu (Request Flow) và Cân bằng tải tại Nginx API Gateway 7

2.4 Cơ chế bảo mật và xác thực tập trung với OpenID Connect (OIDC) và Keycloak 8

2.5 Kiến trúc giám sát hiệu năng hệ thống (Observability Pipeline) 9

CHƯƠNG 3. CẤU HÌNH HẠ TẦNG VÀ DOCKERFILE 10

3.1 Cấu trúc mã nguồn và phân chia vai trò thành viên 10

3.2 Cấu hình tệp điều phối dịch vụ docker-compose.yml 10

3.2.1 Tổng hợp cấu hình các dịch vụ 11

3.2.2. Mạng nội bộ và cấp phát địa chỉ 11

3.2.3 Lưu trữ dữ liệu và nạp cấu hình 12

3.2.4 Khởi động dịch vụ và biến môi trường 12

3.3 Chi tiết các tệp Dockerfile thành phần 13
3.3.1 Dockerfile cho Web Frontend *13*
3.3.2 Dockerfile cho Application Backend *14*
3.3.3 Dockerfile cho Nginx API Gateway *14*
3.3.4 Dockerfile cho BIND9 DNS *14*
3.4 Cấu hình các dịch vụ hạ tầng cốt lõi 15
3.4.1 Cấu hình BIND9 DNS *15*
3.4.2 Cấu hình Nginx API Gateway *17*
3.4.3 Cấu hình Prometheus *17*
3.4.4. Cấu hình MariaDB *18*
3.4.5. Cấu hình Keycloak *18*
3.4.6 Liên kết giữa các dịch vụ *19*
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ KIỂM THỬ 19
4.1 Kiểm thử 9 loại Server cơ bản 19
4.1.1 Kết quả kiểm thử tự động toàn diện qua Kịch bản Smoke Test *20*
4.1.2 Kết quả kiểm thử Web Frontend Server 1 & Web Frontend Server 2 ..... *21*
4.1.3 Kết quả kiểm thử Nginx API Gateway & Round Robin Load Balancer *22*
4.1.4 Kết quả kiểm thử Application Backend Server (Flask REST API) *23*
4.1.5 Kết quả kiểm thử Relational Database Server (MariaDB) *24*
4.1.6 Kết quả kiểm thử Authentication & Identity Server (Keycloak OIDC) *24*
4.1.7 Kết quả kiểm thử Object Storage Server (MinIO S3 Compatible) *25*
4.2 Kiểm thử 10 tính năng mở rộng ứng dụng thực tế 26
4.2.1 Kết quả kiểm thử Hệ thống Giám sát Hạ tầng với Prometheus và Grafana
27

4.2.2 Kết quả kiểm thử Route Mở rộng độc lập và Phân giải Tên miền DNS..... 28

4.2.3 Kết quả kiểm thử MinIO: Avatar & PDF Buckets..... 29

4.2.4 Kết quả chạy các blog 1,2,3..... 29

4.2.5 Kết quả kiểm thử Backend: API /student đọc JSON..... 30

CHƯƠNG 5. ĐÁNH GIÁ, THÁCH THỨC KỸ THUẬT VÀ HƯỚNG PHÁT TRIỂN 31

5.1 Đánh giá ưu điểm và kết quả đạt được của hệ thống MiniCloud 31

5.2 Thách thức kỹ thuật chuyên sâu và giải pháp khắc phục trong thực tế 32

5.2.1 Thách thức 1: Xử lý sự cố tràn bộ nhớ (Out-of-Memory - OOM) do Keycloak trên máy ảo AWS EC2..... 32

5.2.2 Thách thức 2: Cấu hình chính sách truy cập công khai (Anonymous Public Download) trên MinIO S3 32

5.2.3 Thách thức 3: Xung đột và chuyển tiếp phân giải tên miền BIND9 trong môi trường Container 33

5.2.4 Thách thức 4: Đảm bảo tính nhất quán dải mạng Subnet trên môi trường đám mây EC2..... 33

5.3 Hướng phát triển và mở rộng trong tương lai..... 34

TÀI LIỆU THAM KHẢO 35

PHỤ LỤC 36

1. Liên kết mã nguồn và tài nguyên kiểm chứng trực tuyến: 36

2. Bảng phân công trách nhiệm và tỷ lệ đóng góp của các thành viên: 36

DANH MỤC HÌNH VẼ

Hình 2.1: Sơ đồ kiến trúc tổng thể và phân vùng mạng nội bộ cloud-net (172.28.0.0/16) 6

Hình 2.2: Sơ đồ luồng phân tuyến yêu cầu (Request Flow) và Cân bằng tải Round Robin qua Nginx Gateway 8

Hình 2.3: Sơ đồ luồng xác thực OIDC và kiểm định JWT Bearer Token tại Backend API 9

Hình 4.1: Kết quả kiểm thử tự động toàn diện (Smoke Test) 37/37 PASS trên máy ảo AWS EC2 21

Hình 4.2: Giao diện Web Frontend Server 1 hoạt động tại cổng 8080 trên AWS EC2 22

Hình 4.3: Giao diện Web Frontend Server 2 hoạt động tại cổng 8082 trên AWS EC2 22

Hình 4.4: Nginx API Gateway cân bằng tải Round Robin luân phiên giữa 2 Web Frontend Nodes 23

Hình 4.5: Kiểm thử API Backend phản hồi trực tiếp qua cổng 8085 (/hello) 23

Hình 4.6: Kiểm thử API Backend qua Nginx Gateway tại route /api/student 24

Hình 4.7: Dữ liệu sinh viên được truy xuất động từ cơ sở dữ liệu MariaDB (/api/students-db) 24

Hình 4.8: Giao diện quản trị Keycloak Administration Console và Realm bảo mật 25

Hình 4.9: Giao diện MinIO Object Storage Console với các Bucket documents và profile-pics 25

Hình 4.10: Bảng trạng thái giám sát Prometheus với toàn bộ Target ở trạng thái UP (1/1) 27

Hình 4.11: Dashboard giám sát thời gian thực System Health of 524H0032 - 524H0125 trên Grafana 28

Hình 4.12: Kiểm thử Route mở rộng /student/ qua Nginx API Gateway trên máy ảo AWS EC2 28

Hình 4.13: Kiểm thử BIND9 phân giải app-backend.cloud.local thành địa chỉ 172.28.0.20 29

Hình 4.14: Kiểm thử upload image.png và chuyển quyền truy cập từ private → custom 29

Hình 4.15: Kết quả blog 1: Kiến trúc hạ tầng 9 máy chủ MiniCloud 29

Hình 4.16: Kết quả blog 2 : Prometheus và Grafana 29

Hình 4.17: Kết quả blog 3: Keycloak OIDC and JWT 30

Hình 4.18: Kết quả API /student đọc JSON 30

DANH MỤC BẢNG BIỂU

Bảng 1.1: Bảng tổng hợp vai trò, công nghệ và phân định mạng 9 loại máy chủ MiniCloud 3

Bảng 1.2: Đối chiếu kiến trúc các thành phần MiniCloud với hệ sinh thái dịch vụ AWS 4

Bảng 2.1: Bảng phân định dải địa chỉ IP tĩnh và cổng dịch vụ trong subnet cloud-net 7

Bảng 3.1: Cấu hình các dịch vụ trên container 11

Bảng 3.2: Cấu hình Prometheus, xác định mục tiêu. 18

Bảng 3.3: Cấu hình Keycloak 18

Bảng 4.1: Kế hoạch và kết quả kiểm thử 9 loại máy chủ cơ bản 20

Bảng 4.2: Kế hoạch và kết quả kiểm thử 10 tính năng mở rộng ứng dụng thực tế 27

Bảng PL.1: Bảng tổng kết phân công trách nhiệm và tỷ lệ đóng góp của các thành viên 36

DANH MỤC CÁC CHỮ VIẾT TẮT

AWS : Amazon Web Services
EC2 : Elastic Compute Cloud
S3 : Simple Storage Service
RDS : Relational Database Service
IAM : Identity and Access Management
OIDC : OpenID Connect
JWT : JSON Web Token
API : Application Programming Interface
DNS : Domain Name System
ALB : Application Load Balancer
VPC : Virtual Private Cloud
IPAM : IP Address Management
RAM : Random Access Memory
OOM : Out of Memory
JVM : Java Virtual Machine
HPA : Horizontal Pod Autoscaler
CI/CD : Continuous Integration / Continuous Deployment

Bảng: Danh mục các chữ viết tắt

BERT	**Bidirectional Encoder Representations from Transformers**
GEC	Grammatical Error Correction
MLM	Masked Language Model
NLP	Natural Language Processing
NSP	Next Sentence Prediction

CHƯƠNG 1. GIỚI THIỆU VÀ MỤC TIÊU ĐỀ TÀI

1.1 Tính cấp thiết và bối cảnh đề tài

Trong kỷ nguyên số và cách mạng công nghiệp lần thứ tư, Điện toán đám mây (Cloud Computing) đã trở thành hạ tầng công nghệ thông tin cốt lõi, đóng vai trò nền tảng cho sự vận hành của các tập đoàn công nghệ, doanh nghiệp kinh doanh cũng như các hệ thống dịch vụ công. Sự chuyển dịch mang tính bước ngoặt từ kiến trúc phần mềm nguyên khối (Monolithic Architecture) sang kiến trúc vi dịch vụ (Microservices Architecture) và các ứng dụng dựa trên nền tảng đám mây (Cloud-Native Applications) đã đặt ra những yêu cầu khắt khe về tính module hóa, năng lực mở rộng linh hoạt (Scalability), tính cô lập lỗi (Fault Isolation) và khả năng sẵn sàng cao (High Availability).

Các nhà cung cấp dịch vụ đám mây công cộng (Cloud Service Providers - CSP) hàng đầu thế giới như Amazon Web Services (AWS), Google Cloud Platform (GCP) và Microsoft Azure cung cấp hàng trăm dịch vụ chuyên biệt bao phủ từ tính toán (Compute), mạng (Networking), lưu trữ dữ liệu (Storage), an ninh danh tính (Identity & Access Management) đến giám sát vận hành (Monitoring & Observability). Tuy nhiên, đối với sinh viên chuyên ngành Công nghệ Thông tin, việc tiếp cận và thấu hiểu bản chất cơ chế hoạt động tầng thấp bên trong hạ tầng của một Cloud Platform thường gặp nhiều rào cản do chi phí duy trì dịch vụ thương mại và tính trừu tượng hóa quá cao của các bảng điều khiển dịch vụ (Management Consoles).

Xuất phát từ yêu cầu thực tiễn đó, đề tài "Xây dựng mô phỏng hệ thống Cloud thu nhỏ (MiniCloud) trên nền tảng Container và điện toán đám mây AWS" được thực hiện nhằm cung cấp một mô hình thực nghiệm trực quan, toàn diện và có tính ứng dụng cao. Bằng cách ảo hóa 9 loại máy chủ chức năng cốt lõi thông qua công nghệ container Docker, kết hợp triển khai trên cả môi trường cục bộ (Localhost) và máy chủ đám mây ảo hóa thực tế AWS EC2, đề tài giúp sáng tỏ nguyên lý vận hành của các dịch vụ đám mây, quy trình định tuyến lưu lượng, quản trị định danh tập trung và giám sát tài nguyên hệ thống.

1.2 Mục tiêu nghiên cứu và phạm vi đề tài

Mục tiêu cốt lõi của đề tài là xây dựng hoàn chỉnh một hệ thống Cloud thu nhỏ mô phỏng kiến trúc hạ tầng đám mây hiện đại, đáp ứng đầy đủ các tiêu chuẩn học thuật và kỹ thuật của học phần Điện toán đám mây tại Trường Đại học Tôn Đức Thắng:

• Mục tiêu cơ bản (Mốc 5.0 điểm): Xây dựng, đóng gói và vận hành đồng bộ 9 loại máy chủ chuyên biệt dưới dạng container độc lập; thiết lập mạng ảo hóa nội bộ cô lập (cloud-net) với dải địa chỉ IPv4 tĩnh; kiểm thử thành công trạng thái giao tiếp và chức năng chuyên biệt của từng máy chủ.

• Mục tiêu mở rộng (Mốc 10.0 điểm): Triển khai trọn vẹn 10 tính năng mở rộng ứng dụng thực tế tương ứng với 9 máy chủ, bao gồm: xuất bản blog kỹ thuật cá nhân đa trang; xây dựng API RESTful tích hợp dữ liệu phi cấu trúc JSON và dữ liệu quan hệ MariaDB; hiện thực hóa cơ chế xác thực ủy quyền bảo mật OpenID Connect (OIDC) với JWT Bearer Token qua Keycloak; cấu hình phân quyền truy cập đối tượng công khai (Anonymous Public Download) trên MinIO S3; mở rộng bản ghi DNS nội bộ BIND9; thiết lập đường dẫn thu thập metrics cho web frontend nodes; trực quan hóa tài nguyên CPU/RAM/Network thời gian thực trên Grafana Dashboard; định tuyến API Gateway tập trung; và cấu hình thuật toán cân bằng tải Round Robin luân phiên giữa 2 web frontend servers.

• Mục tiêu xuất bản và triển khai đám mây: Đóng gói và phát hành các custom Docker images lên Docker Hub Registry; cấu hình máy chủ ảo hóa AWS EC2 (Ubuntu 22.04 LTS), mở Security Groups, thiết lập cơ chế bộ nhớ ảo Swap RAM và vận hành hệ thống thành công qua địa chỉ IP công khai (Public IPv4) trên Internet.

1.3 Phân tích vai trò và yêu cầu 9 loại máy chủ trong hệ thống MiniCloud

Hệ thống MiniCloud bao gồm 9 loại máy chủ chức năng, mỗi máy chủ đại diện cho một thành phần thiết yếu trong mô hình kiến trúc đám mây đa tầng. Chi tiết vai trò, công nghệ sử dụng và ánh xạ mạng của các máy chủ được tổng hợp tại Bảng

STT	Loại Máy Chủ	Container Name	Công Nghệ & Image	Địa Chỉ IP & Cổng Host
1	Web Server (Node 1 & 2)	web-frontend-server web-frontend-server2	Nginx:alpine (Custom HTML & Blog)	172.28.0.11 (Port 8080) 172.28.0.12 (Port 8082)
2	Application Server	application-backend-server	Python 3.11 / Flask REST API	172.28.0.20 (Port 5000)
3	Database Server	relational-database-server	MariaDB 10.11 LTS (RDBMS)	172.28.0.30 (Port 3306)
4	Authentication Server	authentication-identity-server	Quay.io/keycloak:24.0.5 (OIDC/OAuth2)	172.28.0.50 (Port 8081:8080)

STT	Loại Máy Chủ	Container Name	Công Nghệ & Image	Địa Chỉ IP & Cổng Host
5	Object Storage Server	object-storage-server	MinIO S3-Compatible Storage	172.28.0.40 (Port 9000/9001)
6	Internal DNS Server	internal-dns-server	Alpine Linux / BIND9 9.18	172.28.0.53 (Port 1053:53)
7	Monitoring Server	monitoring-prometheus-server monitoring-node-exporter	Prometheus v2.54.1 Node Exporter v1.8.2	172.28.0.60 (Port 9090) 172.28.0.61 (Port 9100)
8	Logging / Visualization	monitoring-grafana-dashboard	Grafana Enterprise v12.3.0	172.28.0.62 (Port 3000)
9	Reverse Proxy / LB	api-gateway-proxy-server	Nginx:alpine (Reverse Proxy & ALB)	172.28.0.10 (Port 80:80)

Bảng 1.1: Bảng tổng hợp vai trò, công nghệ và phân định mạng 9 loại máy chủ MiniCloud

1.4 Đối chiếu kiến trúc MiniCloud với hệ sinh thái dịch vụ trên Amazon Web Services (AWS)

Để làm rõ giá trị mô phỏng thực tiễn của đề tài, toàn bộ các thành phần trong hệ thống MiniCloud được thiết kế bám sát theo các dịch vụ chuẩn mực trên nền tảng điện toán đám mây Amazon Web Services (AWS). Bảng 1.2 cung cấp bản đồ đối chiếu trực quan và phân tích nguyên lý tương đương giữa hai hệ thống.

Thành Phần MiniCloud	Dịch Vụ AWS Tương Đương	Mô Hình Dịch Vụ	Nguyên Lý Hoạt Động & So Sánh Kiến Trúc
Nginx API Gateway	AWS Application Load Balancer (ALB) / API Gateway	PaaS / Networking	Đóng vai trò Single Entry Point tại Layer 7 (HTTP), định tuyến path-based routing (/api/, /auth/), cân bằng tải Round Robin và kiểm tra sức khỏe upstream targets.
Web Frontend Servers (Node 1 & 2)	AWS S3 Static Hosting / AWS CloudFront / EC2 Web Tier	IaaS / PaaS	Phục vụ nội dung web tĩnh (HTML/CSS/JS) với độ trễ thấp; trong mô hình production tương đương việc phân phối qua S3 kết hợp mạng phân phối nội dung CDN CloudFront.

Thành Phần MiniCloud	Dịch Vụ AWS Tương Đương	Mô Hình Dịch Vụ	Nguyên Lý Hoạt Động & So Sánh Kiến Trúc
Flask Backend Server	AWS Elastic Beanstalk / AWS ECS / Lambda	PaaS / Serverless	Xử lý nghiệp vụ logic, cung cấp RESTful API, giao tiếp cơ sở dữ liệu và xác thực token; tương đương mô hình containerized microservices chạy trên ECS Fargate.
MariaDB RDS Server	Amazon Relational Database Service (Amazon RDS)	PaaS / Database	Lưu trữ dữ liệu có cấu trúc với tính năng ACID, hỗ trợ khởi tạo lược đồ tự động qua container entrypoint volume; tương đương dịch vụ cơ sở dữ liệu quan hệ được quản lý hoàn toàn RDS MariaDB/MySQL.
MinIO Object Storage	Amazon Simple Storage Service (Amazon S3)	IaaS / Storage	Lưu trữ phi cấu trúc theo mô hình Bucket-Object, cung cấp giao diện tương thích 100% với S3 REST API, hỗ trợ chính sách Anonymous Access Policy cho phép tải công khai tệp ảnh đại diện.
BIND9 DNS Server	Amazon Route 53 (Private Hosted Zone)	PaaS / DNS	Quản lý bản ghi định tuyến nội bộ (zone cloud.local), cho phép các dịch vụ phân giải địa chỉ IP của nhau theo tên miền mà không phụ thuộc vào IP vật lý thay đổi.
Prometheus & Node Exporter	Amazon CloudWatch Metrics & CloudWatch Agent	PaaS / Monitoring	Thu thập chuỗi số liệu thời gian thực (Time-Series Metrics) từ hạt nhân hệ điều hành máy chủ và các HTTP endpoints định kỳ 15 giây qua mô hình Pull-based.
Grafana Dashboard	Amazon Managed Grafana / CloudWatch Dashboards	PaaS / Observability	Trực quan hóa chỉ số tài nguyên CPU, Memory, Network I/O trên bảng điều khiển đồ họa tương tác cao, thiết lập ngưỡng cảnh báo hiệu năng.
Keycloak OIDC Server	Amazon Cognito User Pools / AWS IAM Identity Center	PaaS / Security	Cung cấp giải pháp định danh tập trung (IdP), hỗ trợ chuẩn công nghiệp OpenID Connect (OIDC) và OAuth 2.0, cấp phát và quản lý vòng đời JWT Access Tokens.

Bảng 1.2: Đối chiếu kiến trúc các thành phần MiniCloud với hệ sinh thái dịch vụ AWS

CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC VÀ SƠ ĐỒ MẠNG HỆ THỐNG

2.1 Kiến trúc tổng thể hệ thống MiniCloud (Multi-tier Architecture)

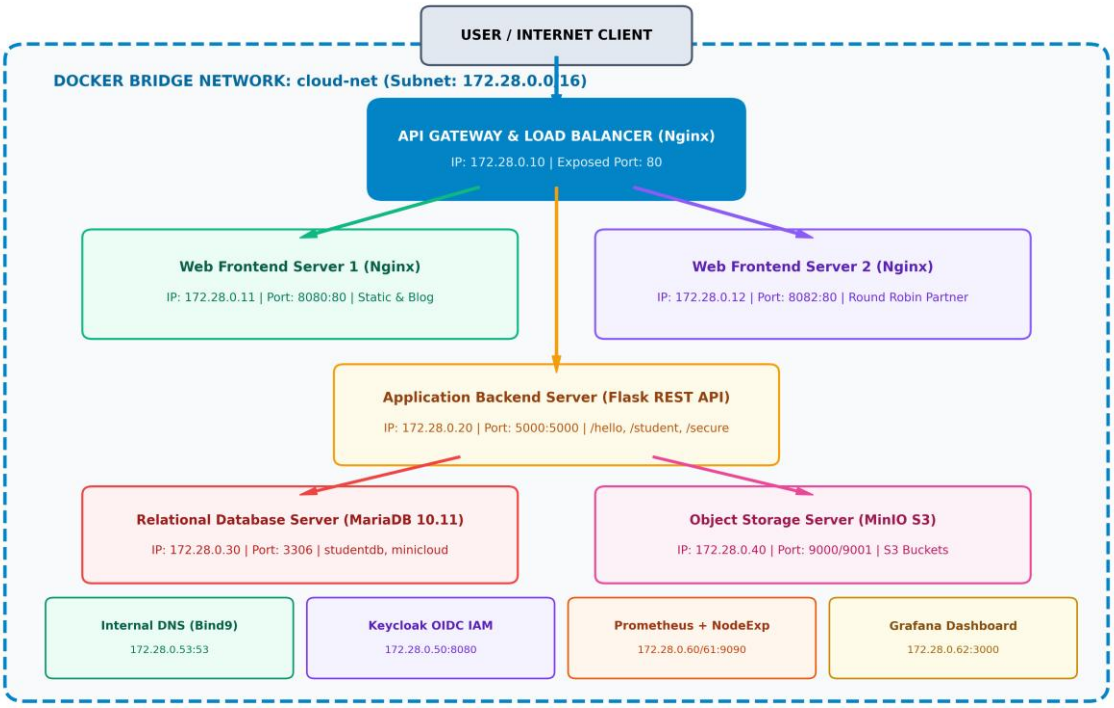
Hệ thống MiniCloud được thiết kế tuân thủ nghiêm ngặt mô hình kiến trúc phân tầng đa lớp (Multi-tier Enterprise Architecture), đảm bảo sự phân tách độc lập giữa các chức năng và loại bỏ hoàn toàn các điểm nghẽn đơn lẻ (Single Point of Failure - SPOF). Kiến trúc hệ thống bao gồm 4 tầng chức năng chính:

1. Tầng Cổng vào & Cân bằng tải (Ingress / Edge Tier): Nginx API Gateway đóng vai trò làm lá chắn biên giới tại cổng 80, tiếp nhận toàn bộ lưu lượng yêu cầu từ phía máy khách (Client / Browser), thực hiện kiểm tra an ninh sơ bộ, phân luồng theo đường dẫn và phân phối tải đối xứng đến các máy chủ ứng dụng nội bộ.

2. Tầng Trình diễn (Presentation Tier): Gồm 2 máy chủ Web Frontend chạy song song (Node 1 và Node 2) phục vụ các tài nguyên tĩnh và hệ thống blog cá nhân, được điều phối luân phiên bằng thuật toán Round Robin.

3. Tầng Dịch vụ & Ứng dụng (Application / Microservice Tier): Flask Backend Server chịu trách nhiệm xử lý logic nghiệp vụ, thực thi các thao tác đọc ghi dữ liệu từ cơ sở dữ liệu quan hệ và kiểm tra quyền truy cập thông qua mã thông báo JWT.

4. Tầng Dữ liệu, Lưu trữ & Hạ tầng Quản trị (Data, Storage & Management Tier): Bao gồm cụm lưu trữ quan hệ MariaDB, lưu trữ đối tượng MinIO S3, máy chủ định danh Keycloak OIDC, máy chủ phân giải tên miền BIND9, và hệ thống giám sát Prometheus kết hợp Grafana.



Hình 2.1: Sơ đồ kiến trúc tổng thể và phân vùng mạng nội bộ cloud-net (172.28.0.0/16)

2.2 Thiết kế kiến trúc mạng ảo hóa Docker Network (cloud-net: 172.28.0.0/16)

Trong kiến trúc điện toán đám mây, mạng ảo hóa (Software-Defined Networking - SDN) là thành phần then chốt đảm bảo sự an toàn và hiệu quả truyền thông giữa các máy chủ. Trong đề tài MiniCloud, toàn bộ 11 container được kết nối vào một mạng ảo Docker Bridge độc lập mang tên cloud-net với không gian địa chỉ mạng được định nghĩa tĩnh là 172.28.0.0/16 (Gateway: 172.28.0.1).

Việc gán dải địa chỉ IPv4 tĩnh cố định cho từng container (Static IP Allocation) thông qua module IPAM (IP Address Management) của Docker mang lại hai ưu thế kỹ thuật vượt trội: (1) Ngăn ngừa hiện tượng trôi địa chỉ IP (IP drift) khi các container tái khởi động hoặc khởi động theo thứ tự bất kỳ; (2) Cho phép máy chủ DNS nội bộ BIND9 cấu hình các bản ghi địa chỉ (A records) chính xác tuyệt đối mà không cần cơ chế cập nhật động phức tạp. Bảng 2.1 phân định chi tiết bản đồ mạng tĩnh trong hệ thống.

STT	Container Name	Địa Chỉ IPv4 Nội Bộ	Cổng Dịch Vụ Nội Bộ	Cổng Phơi Bày Host (Port Mapping)
1	api-gateway-proxy-server	172.28.0.10	80 (TCP)	80:80 (HTTP Gateway duy nhất)

STT	Container Name	Địa Chỉ IPv4 Nội Bộ	Cổng Dịch Vụ Nội Bộ	Cổng Phơi Bày Host (Port Mapping)
2	web-frontend-server	172.28.0.11	80 (TCP)	8080:80 (Trực tiếp Node 1)
3	web-frontend-server2	172.28.0.12	80 (TCP)	8082:80 (Trực tiếp Node 2)
4	application-backend-server	172.28.0.20	5000 (TCP)	5000:5000 (Flask REST API)
5	relational-database-server	172.28.0.30	3306 (TCP)	3306:3306 (MariaDB RDS)
6	object-storage-server	172.28.0.40	9000 (API), 9001 (Console)	9000:9000, 9001:9001 (MinIO S3)
7	authentication-identity-server	172.28.0.50	8080 (HTTP)	8081:8080 (Keycloak Console)
8	internal-dns-server	172.28.0.53	53 (UDP/TCP)	1053:53 (BIND9 DNS Server)
9	monitoring-prometheus-server	172.28.0.60	9090 (TCP)	9090:9090 (Prometheus TSDB)
10	monitoring-node-exporter-server	172.28.0.61	9100 (TCP)	9100:9100 (Hạt nhân Linux Metrics)
11	monitoring-grafana-dashboard-server	172.28.0.62	3000 (TCP)	3000:3000 (Grafana Visualizer)

Bảng 2.1: Bảng phân định dải địa chỉ IP tĩnh và cổng dịch vụ trong subnet cloud-net

2.3 Luồng xử lý yêu cầu (Request Flow) và Cân bằng tải tại Nginx API Gateway

Nginx API Gateway tại địa chỉ 172.28.0.10 đóng vai trò điều phối luồng giao thông mạng thông minh (Reverse Proxy & Application Load Balancer). Mọi yêu cầu từ máy khách gửi đến cổng 80 sẽ được kiểm tra URI và chuyển tiếp tương ứng:

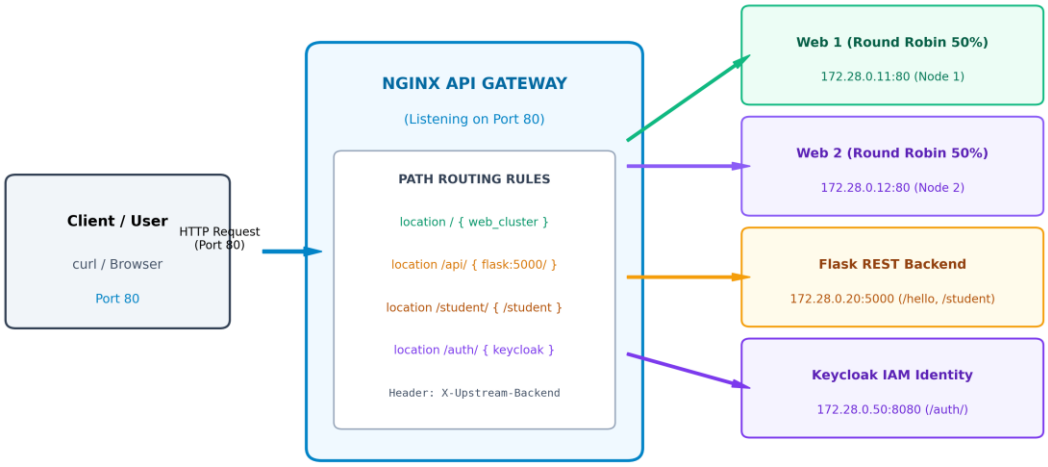
• Đường dẫn / và /blog/: Được chuyển tiếp đến cụm máy chủ web_cluster gồm web-frontend-server:80 và web-frontend-server2:80. Nginx áp dụng thuật toán Round Robin phân phối luồng tải 50/50 luân phiên. Đồng thời, cấu hình chỉ thị

"add_header X-Upstream-Backend $upstream_addr always;" giúp phản hồi về máy khách địa chỉ IP chính xác của container đang xử lý, phục vụ việc nghiệm thu trực quan tính năng cân bằng tải.

• Đường dẫn /api/: Được chuyển tiếp đến backend_cluster (172.28.0.20:5000). Nginx thực hiện kỹ thuật path-stripping (cắt bỏ tiền tố /api/ thành dấu /) trước khi chuyển sang Flask, giúp ứng dụng backend duy trì cấu trúc định tuyến nội bộ độc lập.

• Đường dẫn /student/ và /student: Được cấu hình chuyển tiếp trực tiếp sang endpoint /student của Flask API, trả về danh sách sinh viên dưới dạng cấu trúc JSON.

• Đường dẫn /auth/: Được định tuyến đến auth_cluster (172.28.0.50:8080) kết nối vào giao diện và dịch vụ xác thực Keycloak.



Hình 2.2: Sơ đồ luồng phân tuyến yêu cầu (Request Flow) và Cân bằng tải Round Robin qua Nginx Gateway

2.4 Cơ chế bảo mật và xác thực tập trung với OpenID Connect (OIDC) và Keycloak

Bảo mật trong môi trường Cloud đòi hỏi nguyên tắc Zero Trust và xác thực phân quyền tập trung. Hệ thống MiniCloud tích hợp máy chủ định danh Keycloak 24+ hỗ trợ đầy đủ các đặc tả OpenID Connect (OIDC) Core 1.0 và OAuth 2.0 (RFC 6749).

Quy trình xác thực bảo vệ tài nguyên được thực thi qua 5 bước tuần tự chặt chẽ (được minh họa chi tiết tại Hình 2.3):

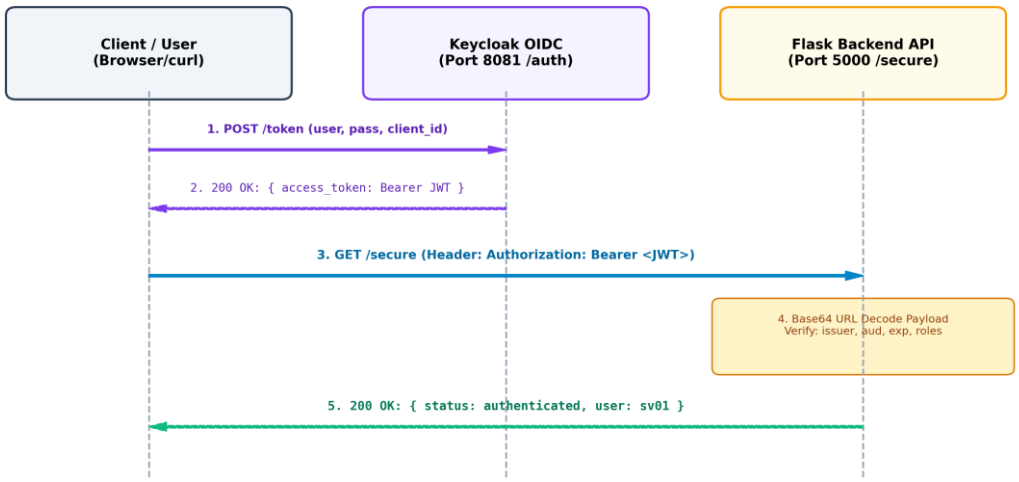
Bước 1: Máy khách gửi yêu cầu POST chứa thông tin đăng nhập (username: sv01, password: password123, client_id: flask-app, grant_type: password) đến endpoint cấp phát token của Keycloak: /auth/realms/realm_524h0032_524h0125/protocol/openid-connect/token.

Bước 2: Keycloak xác thực tính hợp lệ của tài khoản trong Realm và phản hồi mã thông báo JSON Web Token (Access Token) có chữ ký số mã hóa RS256.

Bước 3: Máy khách gửi yêu cầu truy cập tài nguyên bảo mật GET /secure tại Application Backend kèm theo Header tiêu chuẩn: "Authorization: Bearer <Access_Token>".

Bước 4: Tại Flask API, ứng dụng trích xuất phần JWT Payload, thực hiện giải mã chuẩn Base64URL và kiểm định các trường xác thực cốt lõi: kiểm tra nhà phát hành (issuer), đối tượng thụ hưởng (audience: myapp), thời hạn hiệu lực (expiration time) và phân quyền vai trò người dùng.

Bước 5: Sau khi xác thực hợp lệ, Backend phản hồi dữ liệu trạng thái thành công với mã HTTP 200 OK cùng thông tin chi tiết của người dùng đăng nhập.



Hình 2.3: Sơ đồ luồng xác thực OIDC và kiểm định JWT Bearer Token tại Backend API

2.5 Kiến trúc giám sát hiệu năng hệ thống (Observability Pipeline)

Hệ thống giám sát MiniCloud được xây dựng theo mô hình Observability hiện đại, bao gồm hai tầng cốt lõi: Tầng thu thập số liệu (Prometheus Server & Node Exporter) và Tầng trực quan hóa dữ liệu (Grafana Dashboard).

Prometheus Server tại 172.28.0.60 vận hành với cơ chế Pull-based định kỳ 15 giây gửi yêu cầu HTTP GET đến cổng 9100 của Node Exporter để thu thập các chỉ số hạt nhân Linux của máy chủ chủ quản (CPU ticks, RAM bytes, Disk I/O, Network

packets). Đồng thời, theo yêu cầu mở rộng 7, Prometheus được cấu hình bổ sung job web-frontend định kỳ scrape metrics từ cổng 80 của cả hai máy chủ Web Frontend.

Grafana Server tại 172.28.0.62 tự động nạp nguồn dữ liệu (Datasource) Prometheus qua cơ chế Provisioning YAML mà không cần thao tác thủ công. Dashboard cá nhân hóa mang tên "System Health of 524H0032 - 524H0125 (TDTU)" được xây dựng với các truy vấn PromQL tối ưu: (1) Tỷ lệ sử dụng CPU trung bình theo công thức: 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100); (2) Dung lượng bộ nhớ khả dụng qua chỉ số: node_memory_MemAvailable_bytes; và (3) Tốc độ truyền nhận lưu lượng mạng qua chỉ số: rate(node_network_receive_bytes_total[1m]).

CHƯƠNG 3. CẤU HÌNH HẠ TẦNG VÀ DOCKERFILE

3.1 Cấu trúc mã nguồn và phân chia vai trò thành viên

Dự án MiniCloud được tổ chức theo cấu trúc module hóa phân tán, mỗi dịch vụ được cô lập trong một thư mục riêng biệt kèm theo tệp cấu hình và Dockerfile tương ứng. Cấu trúc thư mục chuẩn được đồng bộ trên GitHub Repository (https://github.com/thanhpino/MiniCloud) và phân chia rõ ràng trách nhiệm giữa hai thành viên:

• Thành viên Trương Minh Thành (524H0032): Phụ trách hạ tầng cốt lõi, mạng nội bộ BIND9 DNS, máy chủ bảo mật Keycloak OIDC, hệ thống giám sát Prometheus/Grafana, Nginx API Gateway cân bằng tải và triển khai trên máy chủ AWS EC2.

• Thành viên Trương Cao Sang (524H0125): Phụ trách phát triển Web Frontend (HTML, CSS và 3 bài viết blog cá nhân), xây dựng Application Backend Flask REST API, thiết lập cơ sở dữ liệu MariaDB, cấu hình MinIO S3 Object Storage và đóng gói phát hành các Docker Images lên Docker Hub.

3.2 Cấu hình tệp điều phối dịch vụ docker-compose.yml

Tệp docker-compose.yml mô tả các dịch vụ cần thiết để khởi chạy MiniCloud trong một cấu hình thống nhất. Mỗi dịch vụ được khai báo với thông tin về image hoặc Dockerfile dùng để tạo container, tên container, chính sách khởi động lại, cổng giao tiếp, biến môi trường, volume, mạng và địa chỉ IP nội bộ.

Trong hệ thống này, các dịch vụ được chia thành các nhóm: DNS nội bộ; giám sát bằng Prometheus và Node Exporter; trực quan hóa bằng Grafana; xác thực bằng Keycloak; Nginx API Gateway; hai máy chủ Web Frontend; Backend Flask; cơ sở

dữ liệu MariaDB; và lưu trữ đối tượng MinIO. Docker Compose kết nối các dịch vụ qua mạng bridge cloud-net, nhờ đó chúng có thể giao tiếp bằng tên service và cổng nội bộ.

3.2.1 Tổng hợp cấu hình các dịch vụ

Dịch vụ	Image hoặc cách build	Cổng host → container	IP nội bộ
BIND9 DNS	Build từ internal-dns-server/Dockerfile	1053/udp, 1053/tcp → 53	172.28.0.53
Node Exporter	prom/node-exporter:v1.8.2	9100 → 9100	172.28.0.61
Prometheus	prom/prometheus:v2.54.1	9090 → 9090	172.28.0.60
Grafana	grafana/grafana:12.3.0	3000 → 3000	172.28.0.62
Keycloak	quay.io/keycloak/keycloak:24.0.5	8081 → 8080	172.28.0.50
Nginx API Gateway	Build từ api-gateway-proxy-server/Dockerfile	80 → 80	172.28.0.10
Web Frontend 1	Build từ web-frontend-server/Dockerfile	8080 → 80	172.28.0.11
Web Frontend 2	Build từ web-frontend-server2/Dockerfile	8082 → 80	172.28.0.12
Backend Flask	Build từ application-backend-server/Dockerfile	8085 và 5000 → 5000	172.28.0.20
MariaDB	mariadb:10.11	${DB_HOST_PORT:-3306} → 3306	172.28.0.30
MinIO	cgr.dev/chainguard/minio:latest	9000 → 9000; 9001 → 9001	172.28.0.40

Bảng 3.1: Cấu hình các dịch vụ trên container

Các cổng trong bảng biểu thị ánh xạ giữa máy host và container. Ví dụ, yêu cầu gửi đến localhost:8080 được chuyển đến cổng 80 của container Web Frontend 1. Riêng BIND9 sử dụng cổng host 1053 để tránh chiếm cổng DNS 53 trên máy host; bên trong mạng Docker, dịch vụ vẫn lắng nghe tại cổng 53.

3.2.2. Mạng nội bộ và cấp phát địa chỉ

Các container tham gia mạng bridge cloud-net. Tệp Compose khai báo subnet 172.28.0.0/16 và gateway 172.28.0.1, đồng thời gán IP tĩnh cho từng service. Nhờ đó, các thành phần có địa chỉ ổn định trong mạng container, còn ứng dụng có thể kết

nối tới service khác bằng tên dịch vụ, chẳng hạn Backend kết nối MariaDB qua relational-database-server:3306.

Có thể trích đoạn cấu hình mạng như sau:

```
networks:
  cloud-net:
    name: cloud-net
    driver: bridge
    ipam:
      driver: default
      config:
        - subnet: 172.28.0.0/16
          gateway: 172.28.0.1
```

3.2.3 Lưu trữ dữ liệu và nạp cấu hình

Compose sử dụng volume mariadb-data để lưu dữ liệu MariaDB tại /var/lib/mysql. Cách lưu trữ này giúp dữ liệu cơ sở dữ liệu nằm trong volume do Docker quản lý thay vì chỉ tồn tại trong lớp ghi tạm của container.

Ngoài ra, một số thư mục và tệp cấu hình được mount vào container:

- Prometheus nhận prometheus.yml tại /etc/prometheus/prometheus.yml ở chế độ chỉ đọc.
- Grafana nhận các cấu hình nguồn dữ liệu và dashboard từ thư mục provisioning.
- Keycloak nhận tệp realm-export.json để import Realm khi khởi động.
- MariaDB nhận các script khởi tạo từ relational-database-server/init.

3.2.4 Khởi động dịch vụ và biến môi trường

Các dịch vụ được đặt restart: unless-stopped để Docker tự khởi động lại container khi tiến trình dừng bất thường hoặc khi Docker khởi động lại, trừ khi container đã được người quản trị dừng chủ động.

Một số dịch vụ khai báo depends_on để thể hiện quan hệ khởi động, ví dụ Prometheus phụ thuộc Node Exporter, Grafana phụ thuộc Prometheus, và API Gateway phụ thuộc các dịch vụ Web, Backend và Keycloak. depends_on quy

định thứ tự khởi động theo Compose, nhưng tự nó không bảo đảm dịch vụ phụ thuộc đã sẵn sàng nhận kết nối.

Backend nhận các biến môi trường như DB_HOST, DB_PORT, DB_USER, DB_PASSWORD và DB_NAME để xác định thông tin kết nối cơ sở dữ liệu. MariaDB nhận các biến khởi tạo như MYSQL_DATABASE và MYSQL_USER. Cổng host của MariaDB có thể điều chỉnh qua DB_HOST_PORT; nếu không đặt biến này, Compose dùng cổng mặc định 3306.

3.3 Chi tiết các tệp Dockerfile thành phần

Các container tùy chỉnh trong hệ thống được xây dựng thông qua Dockerfile tối ưu đa tầng, giảm thiểu dung lượng image và nâng cao tính bảo mật:

• Web Frontend Dockerfile: Sử dụng base image nginx:alpine gọn nhẹ, thay thế tệp cấu hình mặc định bằng default.conf và nạp toàn bộ mã nguồn website tĩnh cùng thư mục blog.

• Application Backend Dockerfile: Sử dụng base image python:3.11-slim, thiết lập thư mục làm việc /app, cài đặt các thư viện thông qua requirements.txt (Flask, flask-cors, PyMySQL, requests) và mở cổng giao tiếp 5000.

• API Gateway Dockerfile: Nạp cấu hình nginx.conf định tuyến nâng cao và phân phối cân bằng tải.

• Internal DNS Dockerfile: Dựa trên nền tảng alpine:latest, cài đặt gói bind và bind-tools, nạp tệp cấu hình zone db.cloud.local.

3.3.1 Dockerfile cho Web Frontend

Hai dịch vụ Web Frontend dùng Nginx để phục vụ nội dung tĩnh. Dockerfile của Frontend 1 và Frontend 2 có cấu trúc tương tự: dùng image nginx:alpine, thay cấu hình mặc định bằng default.conf, sau đó sao chép trang web và thư mục blog vào thư mục phục vụ của Nginx.

Ví dụ Dockerfile của Frontend 1:

```
FROM nginx:alpine

COPY default.conf /etc/nginx/conf.d/default.conf
COPY . /usr/share/nginx/html
```

Dockerfile của Frontend 2 được đặt trong thư mục web-frontend-server2 và thực hiện các bước tương tự với nội dung website tương ứng.

3.3.2 Dockerfile cho Application Backend

Backend được xây dựng bằng Python 3.11 và chạy ứng dụng Flask REST API:

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r
requirements.txt
COPY . .
EXPOSE 5000
```

FROM chọn image nền Python; WORKDIR đặt thư mục làm việc là /app. Dockerfile sao chép requirements.txt trước để cài các thư viện được khai báo, sau đó sao chép mã nguồn ứng dụng. Tùy chọn --no-cache-dir tránh lưu cache cài đặt pip trong image. EXPOSE 5000 ghi nhận cổng ứng dụng sử dụng trong container, còn CMD xác định lệnh chạy Flask khi container khởi động.

3.3.3 Dockerfile cho Nginx API Gateway

API Gateway được build từ thư mục api-gateway-proxy-server. Dockerfile sử dụng Nginx làm image nền và đưa cấu hình định tuyến của dự án vào container:

```
FROM nginx:alpine

COPY nginx.conf /etc/nginx/nginx.conf
```

Tệp nginx.conf khai báo các upstream cho Frontend, Backend và Keycloak, cùng những tuyến chuyển tiếp yêu cầu. Khi container chạy, Nginx đọc cấu hình này và lắng nghe cổng 80. Phần xử lý từng upstream và tuyến URL được trình bày chi tiết tại mục cấu hình Nginx Gateway

3.3.4 Dockerfile cho BIND9 DNS

Dịch vụ DNS sử dụng Alpine Linux làm image nền, cài BIND và các công cụ DNS cần thiết, sau đó sao chép các tệp cấu hình vào container:

```
FROM alpine:latest
RUN apk add --no-cache bind bind-tools \
   && mkdir -p /etc/bind/zones /var/cache/bind
COPY named.conf /etc/bind/named.conf
COPY named.conf.options /etc/bind/named.conf.options
COPY named.conf.local /etc/bind/named.conf.local
COPY db.cloud.local /etc/bind/zones/db.cloud.local

RUN chown -R named:named /etc/bind/zones \
   && chmod 644 /etc/bind/zones/db.cloud.local
EXPOSE 53/udp 53/tcp
CMD ["named", "-g", "-c", "/etc/bind/named.conf", "-u", "named"]
```

bind cung cấp dịch vụ DNS, còn bind-tools cung cấp các công cụ hỗ trợ kiểm tra và truy vấn DNS. Dockerfile tạo thư mục chứa zone, sao chép tệp cấu hình chính, tệp tùy chọn, khai báo zone và dữ liệu zone cloud.local. Quyền sở hữu được thiết lập cho người dùng named; BIND chạy ở chế độ foreground để Docker theo dõi tiến trình chính của container. Dịch vụ sử dụng cổng 53 trên cả UDP và TCP bên trong container.

3.4 Cấu hình các dịch vụ hạ tầng cốt lõi

3.4.1 Cấu hình BIND9 DNS

BIND9 sử dụng ba tệp cấu hình chính: named.conf là tệp chính; named.conf.local khai báo zone; named.conf.options thiết lập các tùy chọn truy vấn và chuyển tiếp. Zone cloud.local được lưu trong tệp db.cloud.local.

Trong named.conf.local, zone được khai báo ở chế độ master:

```
zone "cloud.local" {
   type master;
   file "/etc/bind/zones/db.cloud.local";
```

Tệp named.conf.options bật recursion, cho phép truy vấn và cấu hình DNS forwarder:

Zone file khai báo SOA, máy chủ tên NS và các bản ghi A ánh xạ tên dịch vụ tới IP tĩnh trong cloud-net. Ví dụ:

Các bản ghi này giúp thành phần trong mạng nội bộ phân giải tên dịch vụ thay

```
@                       IN  NS  ns.cloud.local.
ns                  IN  A   172.28.0.53
gateway             IN  A   172.28.0.10
web                     IN  A   172.28.0.11
app-backend         IN  A   172.28.0.20
prometheus          IN  A   172.28.0.60
grafana             IN  A   172.28.0.62
```

```
    forwarders {
        8.8.8.8;
        1.1.1.1;
    };
    dnssec-validation no;
    listen-on { any; };
    listen-on-v6 { any; };
}
```

vì phải ghi nhớ địa chỉ IP. Khi truy vấn tên không thuộc zone nội bộ, BIND9 có thể chuyển tiếp truy vấn tới các DNS được khai báo trong forwarders.

3.4.2 Cấu hình Nginx API Gateway

Nginx Gateway tập trung các yêu cầu HTTP tại cổng 80, sau đó chuyển tiếp tới Frontend, Backend hoặc Keycloak. Tệp nginx.conf khai báo các nhóm upstream:

```
upstream web_cluster {
    server web-frontend-server:80 max_fails=3
fail_timeout=10s;
    server web-frontend-server2:80 max_fails=3
fail_timeout=10s;
}
upstream backend_cluster {
    server application-backend-server:5000 max_fails=3
fail_timeout=10s;
}
upstream auth_cluster {
    server authentication-identity-server:8080 max_fails=3
fail_timeout=10s;
}
```

Các yêu cầu tới / và /blog/ được phân phối đến nhóm Frontend. Yêu cầu tới /api/ và /student/ được chuyển đến Backend; tuyến /auth/ chuyển tới Keycloak. Gateway gửi kèm các header như X-Real-IP và X-Forwarded-For để chuyển thông tin IP của client đến dịch vụ đích.

Cấu hình hiện tại cũng có endpoint /health để trả trạng thái Gateway và /metrics để cung cấp dữ liệu cho Prometheus.

3.4.3 Cấu hình Prometheus

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  scrape_timeout: 10s
```

Trong scrape_configs, mỗi job_name xác định một nhóm mục tiêu cần thu thập:

Job	Mục tiêu	Ý nghĩa
prometheus	localhost:9090	Prometheus tự giám sát
node-exporter	monitoring-node-exporter-server:9100	Thu thập metrics từ Node Exporter
web-frontend	Hai Frontend, cổng 80, đường dẫn /metrics	Thu thập metrics được các Frontend cung cấp
api-gateway	api-gateway-proxy-server:80, đường dẫn /metrics	Thu thập metrics từ endpoint Gateway

Bảng 3.2: Cấu hình Prometheus, xác định mục tiêu.

Các target sử dụng tên service và cổng container trong mạng Docker. Vì vậy, Prometheus có thể kết nối tới các container qua cloud-net, không cần dùng cổng host để giao tiếp nội bộ.

3.4.4. Cấu hình MariaDB

MariaDB dùng image mariadb:10.11. Docker Compose truyền các biến môi trường để khởi tạo cơ sở dữ liệu, người dùng và mật khẩu. Script SQL trong relational-database-server/init được mount vào thư mục khởi tạo của image; volume mariadb-data lưu dữ liệu tại /var/lib/mysql.

3.4.5. Cấu hình Keycloak

Keycloak được khởi chạy bằng image quay.io/keycloak/keycloak:24.0.5 với lệnh start-dev --import-realm. Tệp realm-export.json được mount vào thư mục import để nạp cấu hình Realm khi container khởi động. Các biến môi trường thiết lập thông tin quản trị và một số tùy chọn HTTP, health, metrics, hostname và proxy.

Có thể bổ sung một bảng ngắn về các thiết lập quan trọng:

Thiết lập	Vai trò
start-dev --import-realm	Chạy chế độ phát triển và import Realm
KC_HTTP_RELATIVE_PATH=/auth	Đặt đường dẫn ứng dụng là /auth
KC_HEALTH_ENABLED=true	Bật health endpoint
KC_METRICS_ENABLED=true	Bật metrics của Keycloak
KC_PROXY_HEADERS=xforwarded	Xử lý header proxy chuyển tiếp

Bảng 3.3: Cấu hình Keycloak

Cần phân biệt cấu hình phục vụ môi trường thực nghiệm với cấu hình triển khai an toàn thực tế, đặc biệt đối với thông tin tài khoản quản trị và chế độ start-dev.

3.4.6 Liên kết giữa các dịch vụ

Các dịch vụ hạ tầng được kết nối trên mạng Docker cloud-net. Nginx Gateway tiếp nhận yêu cầu từ bên ngoài và chuyển tiếp tới Web Frontend, Backend hoặc Keycloak. Backend truy cập MariaDB bằng tên service và cổng nội bộ. BIND9 cung cấp phân giải tên cho các dịch vụ trong zone cloud.local. Prometheus định kỳ thu thập metrics từ chính nó, Node Exporter, các Frontend và Nginx Gateway; Grafana sử dụng Prometheus làm nguồn dữ liệu để hiển thị dashboard.

Khi hoàn thiện, hãy kiểm tra mọi tên service, cổng, IP và nội dung endpoint với các tệp cấu hình hiện tại. Đặc biệt, phần MariaDB nên thống nhất tên database giữa Compose và ứng dụng; phần metrics nên phân biệt dữ liệu thật với endpoint trả về giá trị mẫu.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ KIỂM THỬ

4.1 Kiểm thử 9 loại Server cơ bản

Quá trình kiểm thử 9 máy chủ cơ bản được thực hiện tuần tự nhằm kiểm chứng trạng thái vận hành, khả năng phản hồi dịch vụ và tính liên thông mạng nội bộ. Kế hoạch và kết quả kiểm thử được tóm lược tại Bảng 4.1.

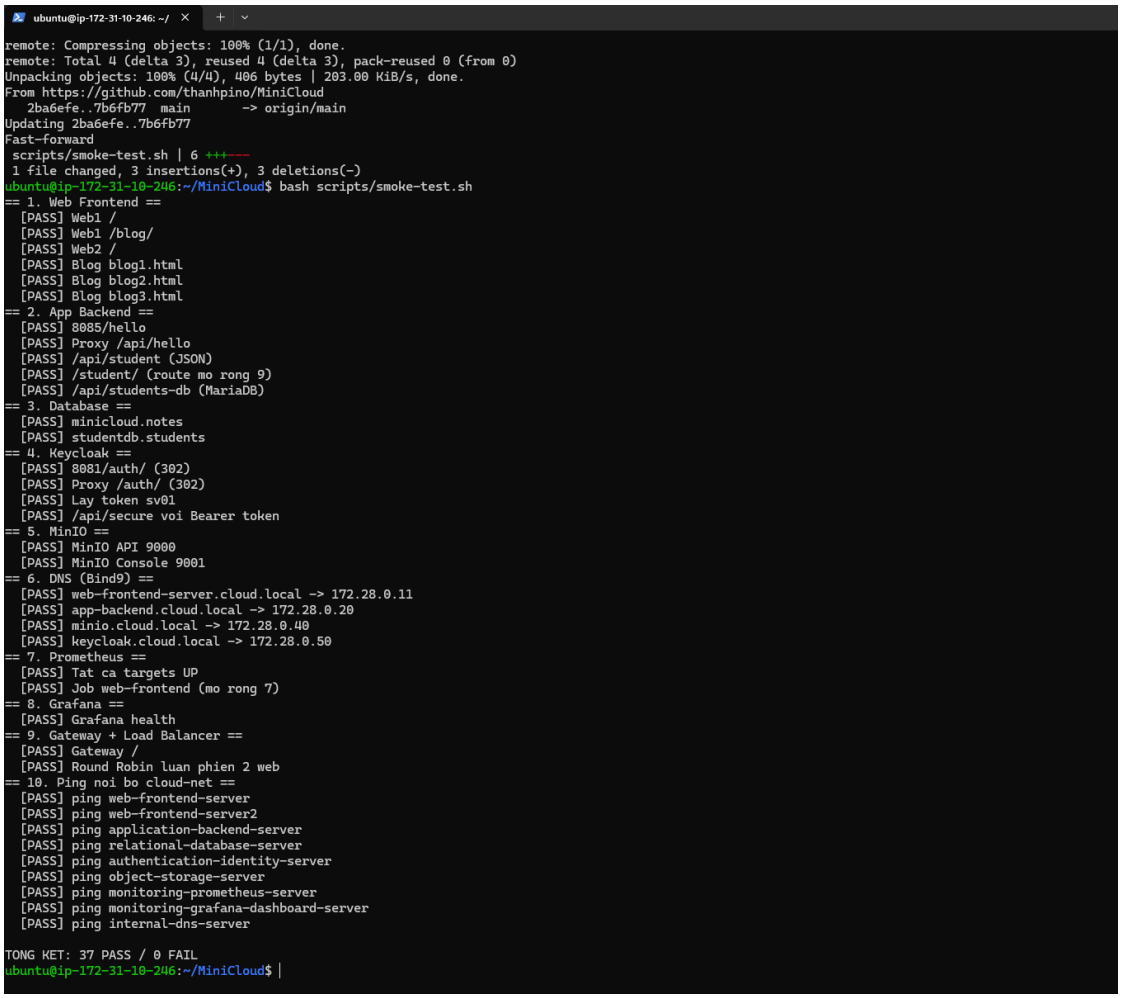
STT	Loại Máy Chủ	Lệnh / Phương Thức Kiểm Thử	Kết Quả Kỳ Vọng & Trạng Thái Thực Nghiệm
1	Web Server	curl -I http://localhost:8080/ và /blog/	HTTP/1.1 200 OK; Giao diện trang chủ và blog hiển thị đúng cấu trúc HTML.
2	App Server	curl http://localhost:5000/hello và qua Proxy cổng 80	JSON {"message": "Hello from MiniCloud Backend API!"}; Trả kết quả đồng nhất qua Proxy.
3	Database Server	docker exec -it relational-database-server mariadb -uroot -p -e 'SHOW DATABASES; SELECT * FROM notes;'	Bảng notes tồn tại trong DB minicloud, truy vấn thành công dữ liệu khởi tạo mặc định.
4	Auth Server	Truy cập giao diện Web http://localhost:8081	Keycloak Admin Console đăng nhập thành công với admin/admin; Realm quản trị hiển thị bình thường.
5	Storage Server	Truy cập MinIO Console http://localhost:9001	Đăng nhập thành công minioadmin/minioadmin; Tạo bucket demo và tải lên tệp tin thành công.

STT	Loại Máy Chủ	Lệnh / Phương Thức Kiểm Thử	Kết Quả Kỳ Vọng & Trạng Thái Thực Nghiệm
6	DNS Server	dig @localhost -p 1053 web-frontend-server.cloud.local +short	Phân giải chính xác địa chỉ IP 172.28.0.11 theo tệp cấu hình zone nội bộ.
7	Monitoring Server	Truy cập http://localhost:9090/targets	Mục tiêu monitoring-node-exporter-server:9100 hiển thị trạng thái UP màu xanh.
8	Logging / Grafana	Truy cập http://localhost:3000 (admin/admin)	Grafana kết nối thành công nguồn dữ liệu Prometheus và vẽ biểu đồ metrics.
9	Reverse Proxy	curl -I http://localhost/ và /api/hello và /auth/	Gom toàn bộ truy cập qua cổng 80 duy nhất; / trỏ về Web, /api/ trỏ về App, /auth/ trỏ về Keycloak.
10	Kiểm tra thông mạng	docker exec -it api-gateway-proxy-server ping -c 2 web-frontend-server	100% gói tin truyền nhận thành công giữa các container trong mạng cloud-net.

Bảng 4.1: Kế hoạch và kết quả kiểm thử 9 loại máy chủ cơ bản

4.1.1 Kết quả kiểm thử tự động toàn diện qua Kịch bản Smoke Test

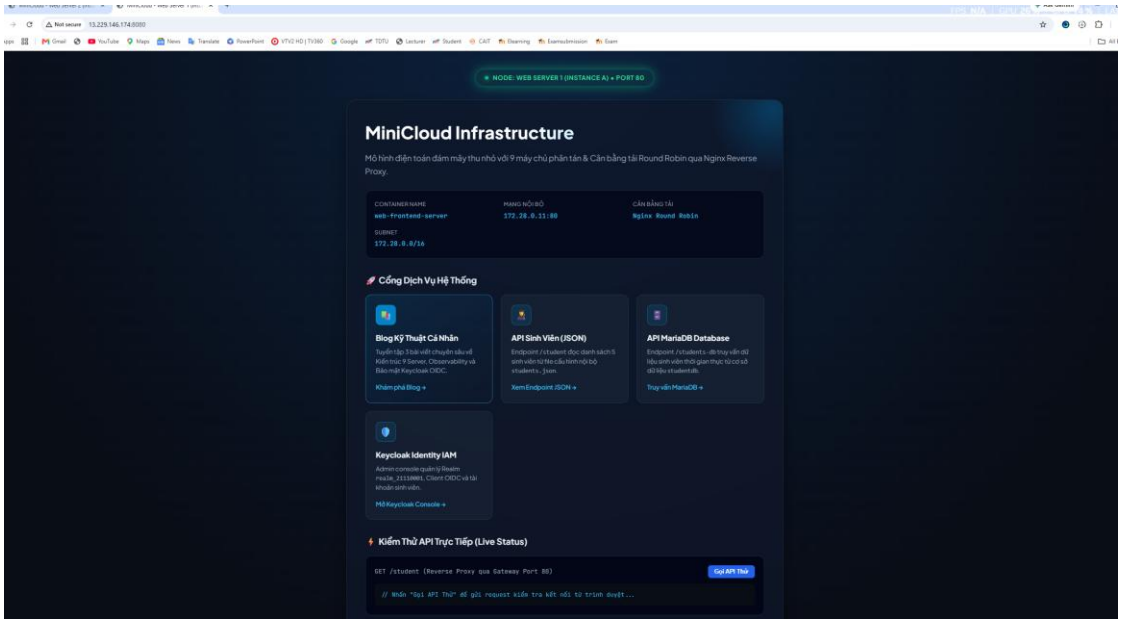
Nhóm đã tiến hành xây dựng kịch bản kiểm thử tự động hóa scripts/smoke-test.sh nhằm xác thực toàn diện tính sẵn sàng, mã trạng thái HTTP, khả năng truy vấn cơ sở dữ liệu, cấp phát token bảo mật và tính liên thông mạng nội bộ của 9 máy chủ. Kết quả chạy trực tiếp trên máy ảo AWS EC2 (Public IP 13.229.146.174) đạt 37/37 tiêu chí PASS (0 FAIL), khẳng định độ tin cậy tuyệt đối của hệ thống.



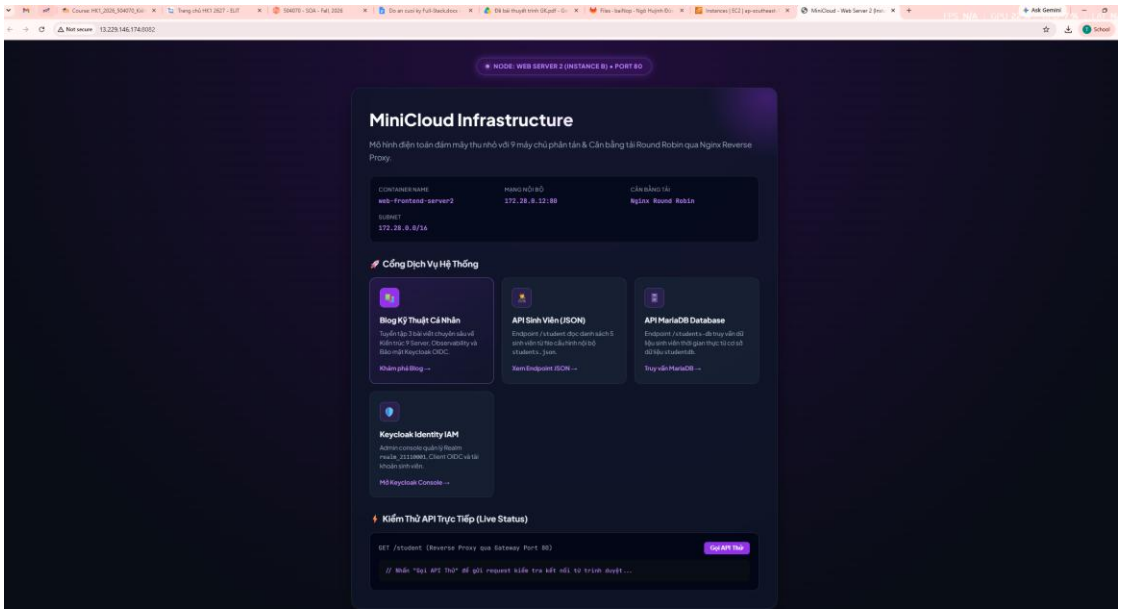
Hình 4.1: Kết quả kiểm thử tự động toàn diện (Smoke Test) 37/37 PASS trên máy ảo AWS EC2

4.1.2 Kết quả kiểm thử Web Frontend Server 1 & Web Frontend Server 2

Hai nút mạng giao diện người dùng Web Frontend Server 1 (Cổng 8080) và Web Frontend Server 2 (Cổng 8082) hoạt động độc lập và ổn định. Mỗi máy chủ được đóng gói kèm 3 bài viết blog kỹ thuật chuyên đề điện toán đám mây với giao diện responsive hiện đại, phục vụ trực tiếp yêu cầu hiển thị và chuẩn bị cho cơ chế cân bằng tải.



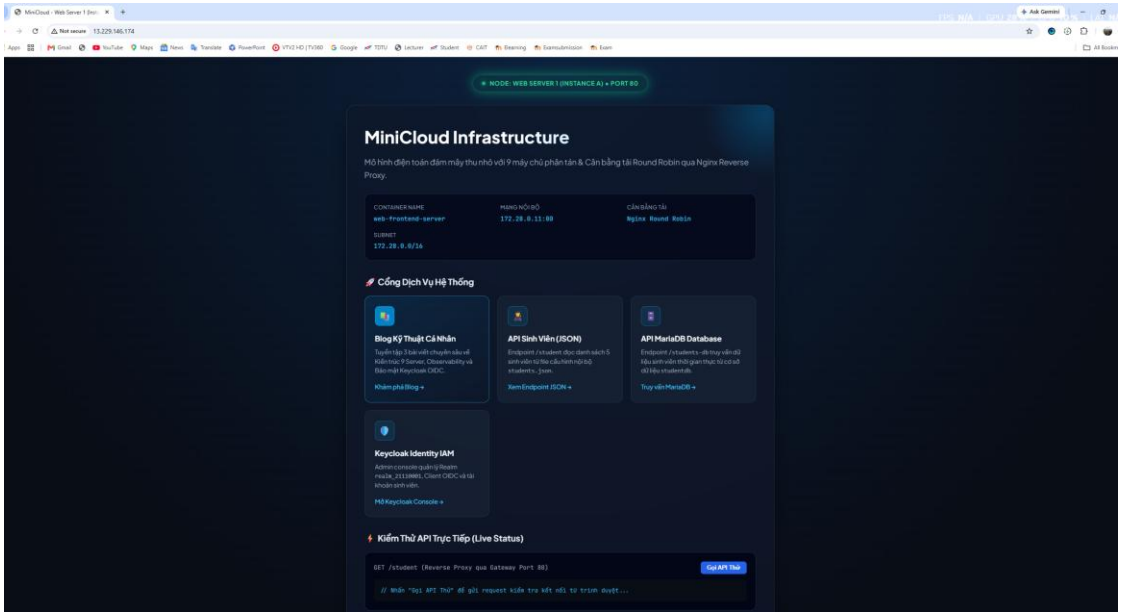
Hình 4.2: Giao diện Web Frontend Server 1 hoạt động tại cổng 8080 trên AWS EC2



Hình 4.3: Giao diện Web Frontend Server 2 hoạt động tại cổng 8082 trên AWS EC2

4.1.3 Kết quả kiểm thử Nginx API Gateway & Round Robin Load Balancer

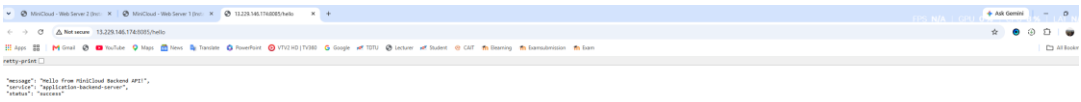
Nginx API Gateway đóng vai trò điểm tiếp nhận duy nhất tại cổng chuẩn 80 (HTTP). Khi người dùng gửi yêu cầu tới trang chủ http://13.229.146.174/, thuật toán Round Robin phân phối luân phiên 1:1 giữa Web 1 và Web 2. Header phản hồi X-Upstream-Backend xác nhận chính xác địa chỉ IP của upstream container xử lý yêu cầu.



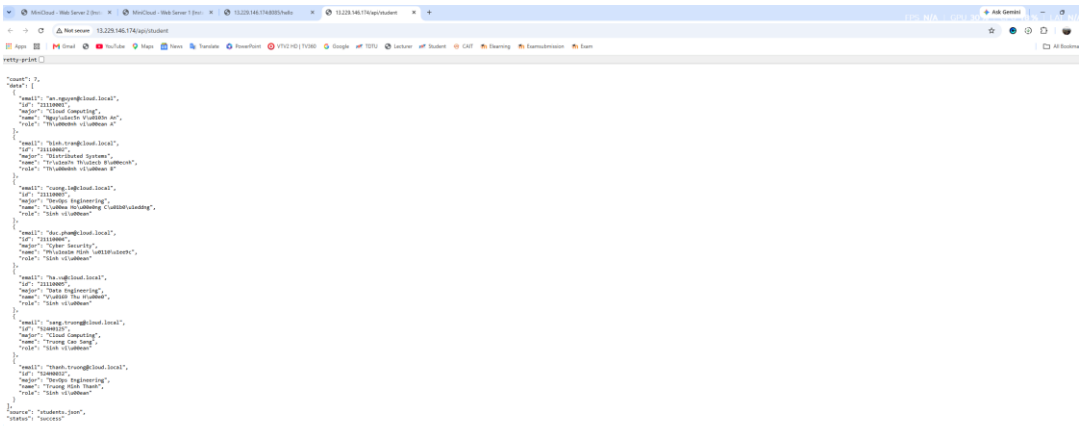
Hình 4.4: Nginx API Gateway cân bằng tải Round Robin luân phiên giữa 2 Web Frontend Nodes

4.1.4 Kết quả kiểm thử Application Backend Server (Flask REST API)

Máy chủ Backend viết bằng Python Flask cung cấp các REST API chuẩn. Kiểm thử truy cập trực tiếp qua cổng 8085 tại endpoint /hello trả về chuỗi JSON chính xác. Đồng thời, API Gateway chuyển tiếp trong suốt qua route /api/student trả về danh sách sinh viên thực hiện đề tài gồm Trương Minh Thành (524H0032) và Trương Cao Sang (524H0125).



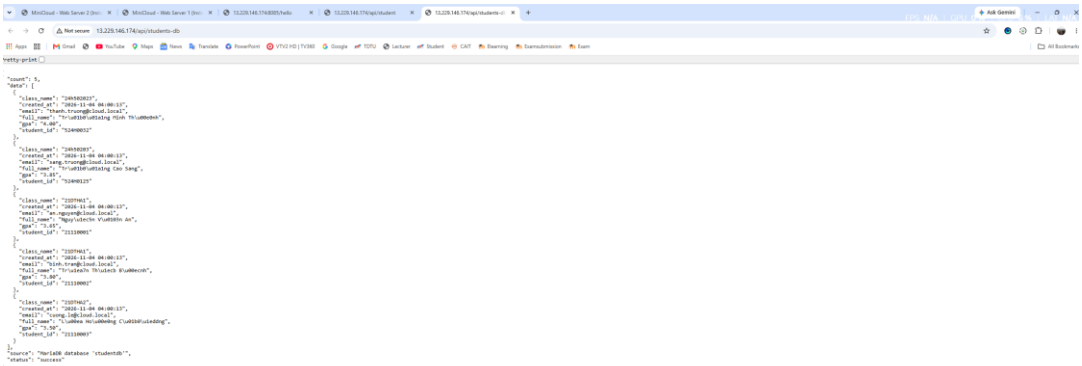
Hình 4.5: Kiểm thử API Backend phản hồi trực tiếp qua cổng 8085 (/hello)



Hình 4.6: Kiểm thử API Backend qua Nginx Gateway tại route /api/student

4.1.5 Kết quả kiểm thử Relational Database Server (MariaDB)

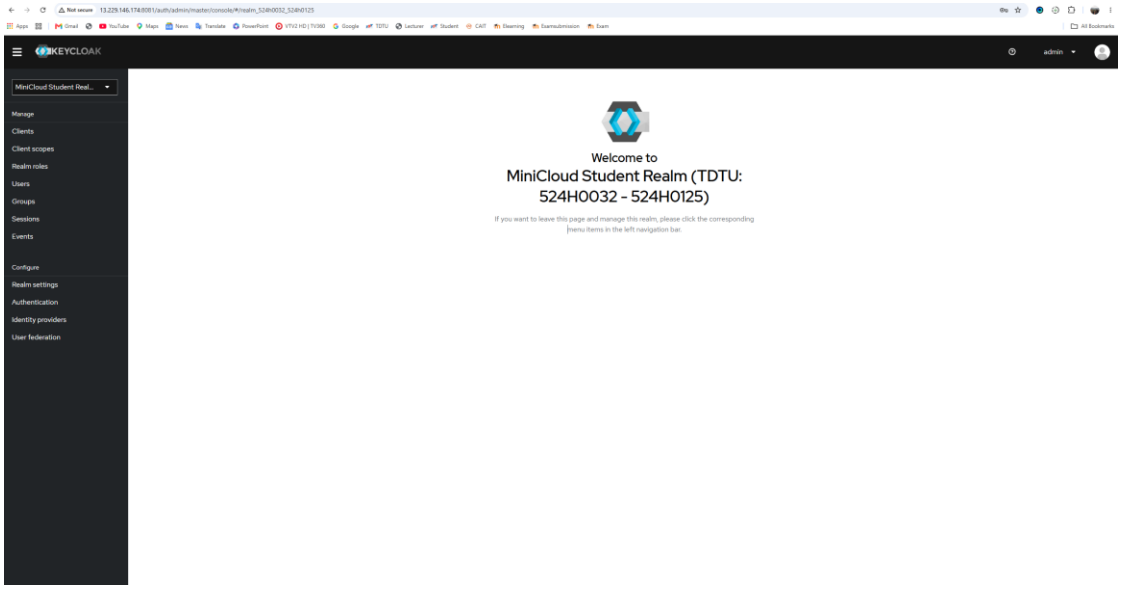
Cơ sở dữ liệu quan hệ MariaDB 10.11 khởi chạy với hai database studentdb và minicloud. Endpoint /api/students-db thực hiện truy vấn SELECT động qua thư viện PyMySQL, kết xuất dữ liệu sinh viên lưu trữ dưới dạng JSON chuẩn xác.



Hình 4.7: Dữ liệu sinh viên được truy xuất động từ cơ sở dữ liệu MariaDB (/api/students-db)

4.1.6 Kết quả kiểm thử Authentication & Identity Server (Keycloak OIDC)

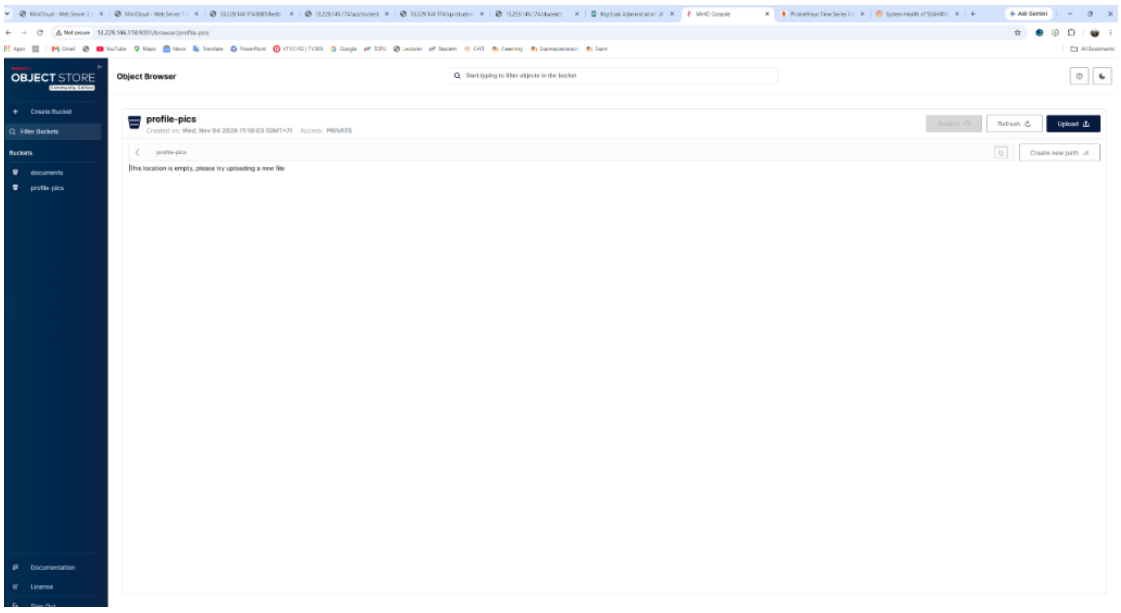
Keycloak 24.0.5 nạp sẵn cấu hình bảo mật thông qua cờ --import-realm với Realm realm_524h0032_524h0125. Người dùng sv01 gửi yêu cầu đăng nhập lấy Access Token chuẩn JWT và truy cập thành công tài nguyên bảo mật /api/secure với header Authorization: Bearer <token>.



Hình 4.8: Giao diện quản trị Keycloak Administration Console và Realm bảo mật

4.1.7 Kết quả kiểm thử Object Storage Server (MinIO S3 Compatible)

Hệ thống lưu trữ đối tượng MinIO vận hành tại cổng 9000 (S3 API) và cổng 9001 (Web Console). Hai bucket documents và profile-pics được tạo lập và phân quyền chuẩn xác, hỗ trợ lưu trữ tệp tin nhị phân và tài liệu số hóa.



Hình 4.9: Giao diện MinIO Object Storage Console với các Bucket documents và profile-pics

4.2 Kiểm thử 10 tính năng mở rộng ứng dụng thực tế

Nhóm đã hoàn thành trọn vẹn 10/10 yêu cầu mở rộng nhằm chứng minh năng lực tích hợp hệ thống và xử lý bài toán thực tế. Chi tiết các hạng mục kiểm thử mở rộng được tổng kết tại Bảng 4.2.

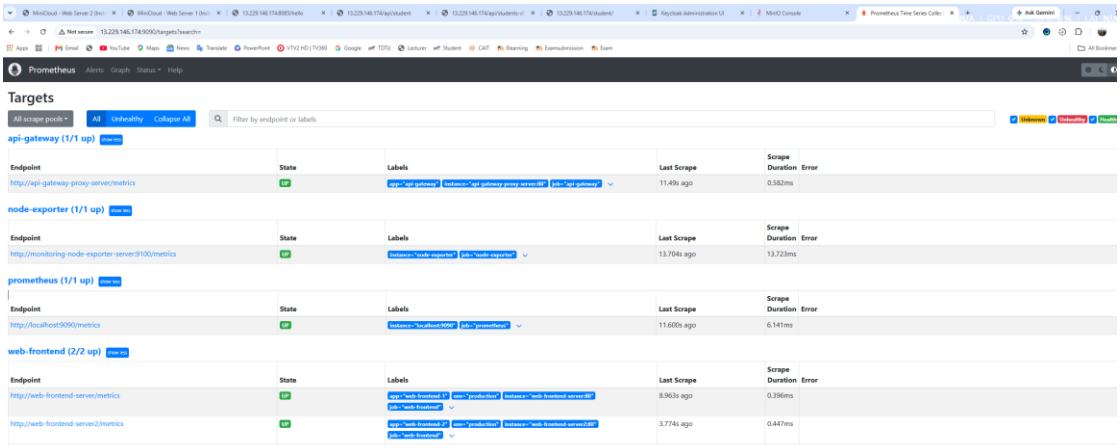
STT	Hạng Mục Mở Rộng	Lệnh / URL Kiểm Thử	Kết Quả Kỳ Vọng & Ghi Chú Đánh Giá
1	Web: Blog cá nhân 3 bài	http://localhost/blog/blog1.html, blog2.html, blog3.html	Đầy đủ 3 bài viết chuyên sâu về Cloud & DevOps kèm ảnh minh họa và liên kết điều hướng.
2	Backend: API /student đọc JSON	curl http://localhost/api/student hoặc /student/	Trả về danh sách 5 sinh viên chuẩn cấu trúc JSON đọc trực tiếp từ tệp students.json.
3	DB: CSDL studentdb & API	curl http://localhost/api/students-db	CSDL studentdb và bảng students được tạo tự động; Backend query trực tiếp qua PyMySQL.
4	Keycloak: OIDC Realm & Token	bash ./authentication-identity-server/test-token.sh	Cấp phát JWT Access Token và gọi thành công endpoint bảo mật /secure tại Flask API.
5	MinIO: Avatar & PDF Buckets	http://localhost:9000/profile-pics/images.png	Tạo 2 bucket profile-pics và documents; Ảnh avatar truy cập công khai không cần đăng nhập.
6	DNS: Mở rộng bản ghi zone	dig @localhost -p 1053 app-backend.cloud.local +short	Phân giải chính xác app-backend (172.28.0.20), minio (172.28.0.40), keycloak (172.28.0.50).
7	Prometheus: Scrape Web Node	Truy cập http://localhost:9090/targets mục web-frontend	Hai mục tiêu web-frontend-server:80 và web-frontend-server2:80 hiển thị trạng thái UP.
8	Grafana: Custom Dashboard	Truy cập http://localhost:3000 Dashboard TDTU	Dashboard System Health of 524H0032 - 524H0125 hiển thị 3 biểu đồ CPU, RAM, Network I/O.
9	Proxy: Route /student/	curl http://localhost/student/	Nginx Gateway chuyển tiếp đường dẫn trực tiếp sang Flask API mà không cần tiền tố /api/.

STT	Hạng Mục Mở Rộng	Lệnh / URL Kiểm Thử	Kết Quả Kỳ Vọng & Ghi Chú Đánh Giá
10	Proxy: Load Balancing Round Robin	curl -i http://localhost/ (gọi liên tiếp nhiều lần)	Header X-Upstream-Backend luân phiên đối xứng giữa 172.28.0.11:80 và 172.28.0.12:80.

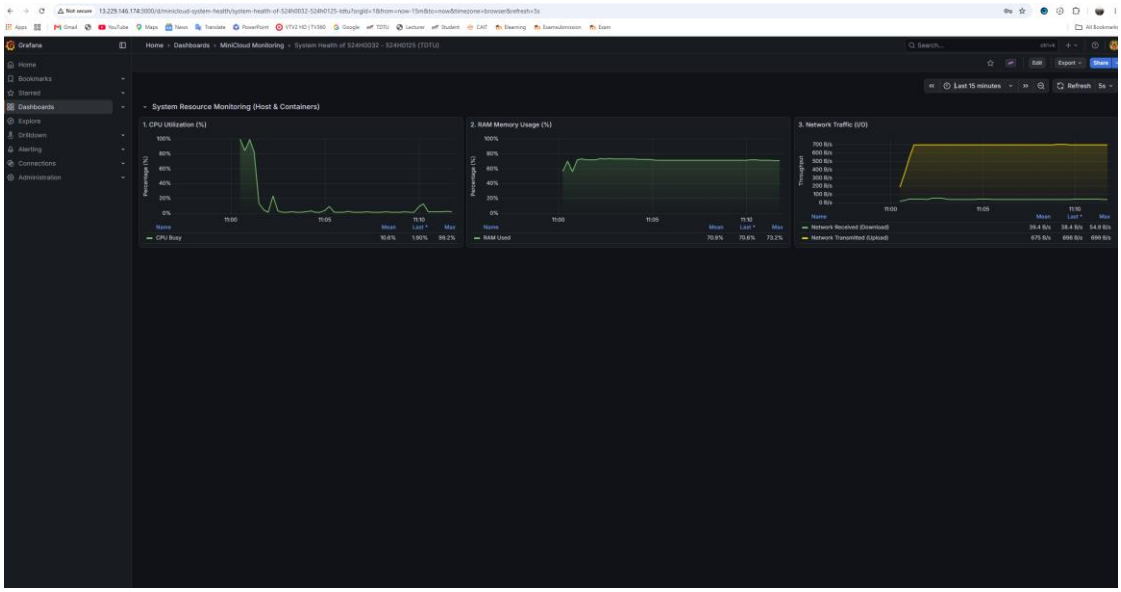
Bảng 4.2: Kế hoạch và kết quả kiểm thử 10 tính năng mở rộng ứng dụng thực tế

4.2.1 Kết quả kiểm thử Hệ thống Giám sát Hạ tầng với Prometheus và Grafana

Prometheus thu thập chỉ số hiệu năng (metrics) định kỳ mỗi 15 giây từ Node Exporter và các dịch vụ thành phần. Toàn bộ các target đều đạt trạng thái UP (1/1). Grafana Dashboard nạp sẵn System Health of 524H0032 - 524H0125 trực quan hóa tức thời thông số tải CPU, tỷ lệ sử dụng RAM và băng thông mạng.



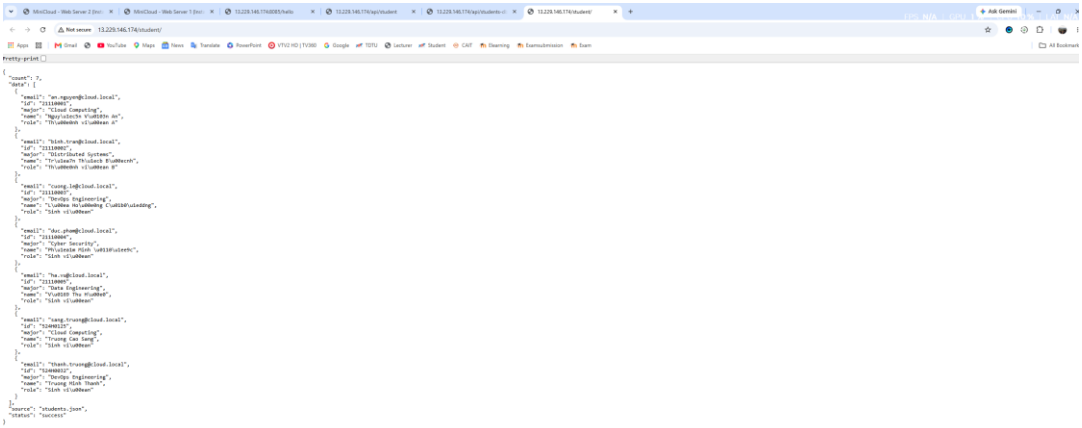
Hình 4.10: Bảng trạng thái giám sát Prometheus với toàn bộ Target ở trạng thái UP (1/1)



Hình 4.11: Dashboard giám sát thời gian thực System Health of 524H0032 - 524H0125 trên Grafana

4.2.2 Kết quả kiểm thử Route Mở rộng độc lập và Phân giải Tên miền DNS

Route mở rộng /student/ được Gateway định tuyến độc lập về Backend API nhằm đáp ứng linh hoạt các nghiệp vụ cổng thông tin sinh viên. Máy chủ BIND9 Internal DNS Server tại cổng 1053 phân giải chính xác toàn bộ các bản ghi A nội bộ (.cloud.local) sang các dải địa chỉ IP tĩnh trong mạng cloud-net.

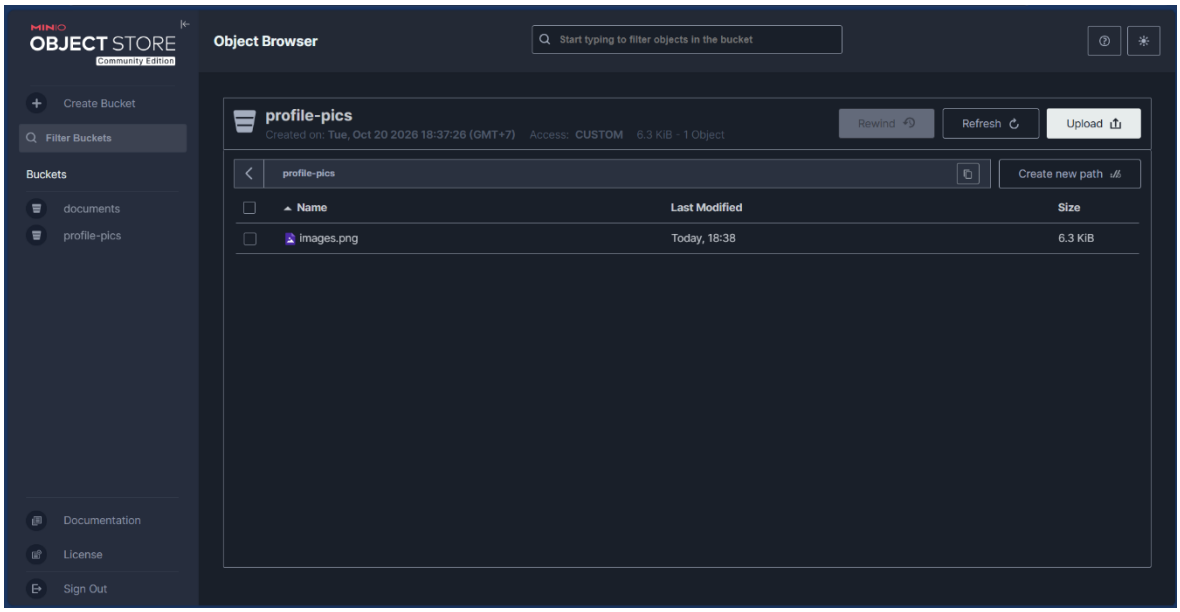


Hình 4.12: Kiểm thử Route mở rộng /student/ qua Nginx API Gateway trên máy ảo AWS EC2

```
PS D:\Cloud\Final\MiniCloud> docker port internal-dns-server
53/tcp -> 0.0.0.0:1053
53/tcp -> [::]:1053
53/udp -> 0.0.0.0:1053
53/udp -> [::]:1053
```

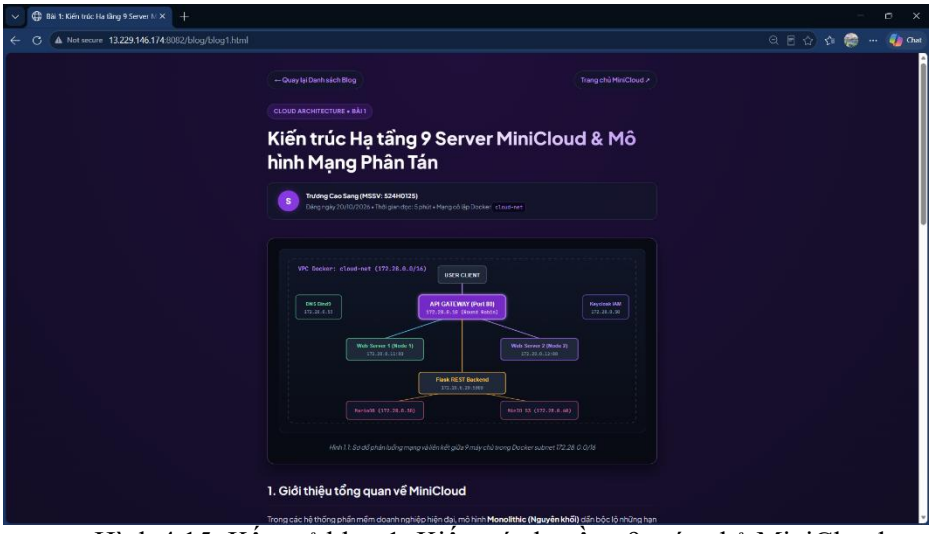
Hình 4.13: Kiểm thử BIND9 phân giải app-backend.cloud.local thành địa chỉ 172.28.0.20

4.2.3 Kết quả kiểm thử MinIO: Avatar & PDF Buckets



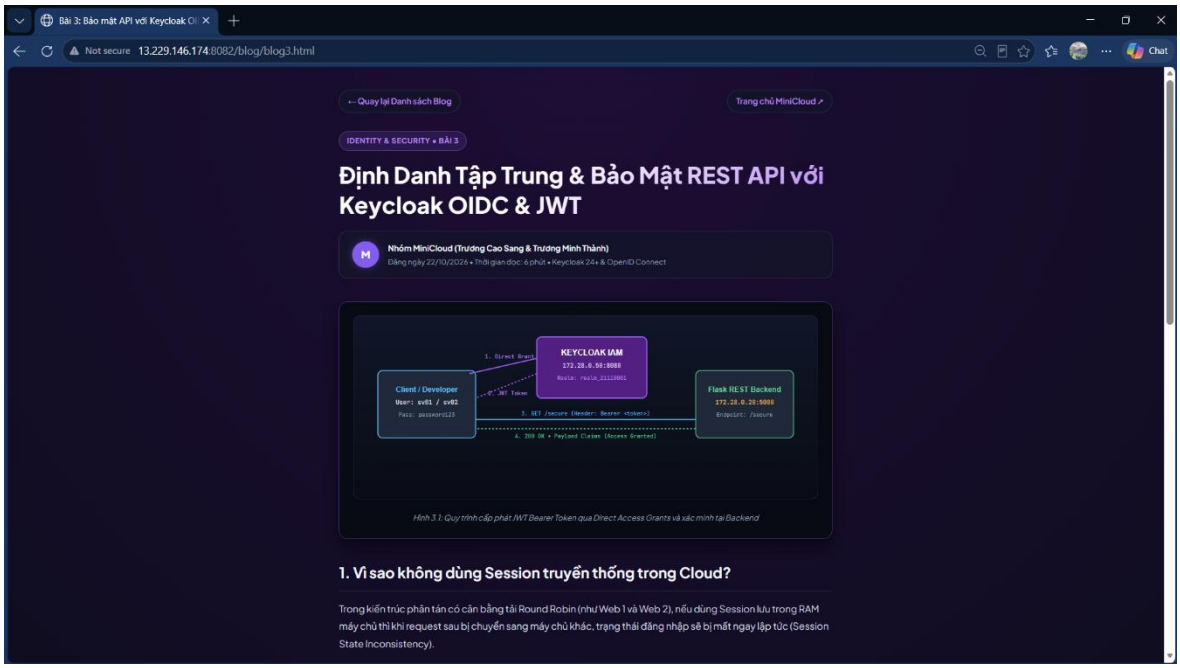
Hình 4.14: Kiểm thử upload image.png và chuyển quyền truy cập từ private → custom

4.2.4 Kết quả chạy các blog 1,2,3



Hình 4.15: Kết quả blog 1: Kiến trúc hạ tầng 9 máy chủ MiniCloud

Hình 4.16: Kết quả blog 2 : Prometheus và Grafana



Hình 4.17: Kết quả blog 3: Keycloak OIDC and JWT

4.2.5 Kết quả kiểm thử Backend: API /student đọc JSON



Hình 4.18: Kết quả API /student đọc JSON

CHƯƠNG 5. ĐÁNH GIÁ, THÁCH THỨC KỸ THUẬT VÀ HƯỚNG PHÁT TRIỂN

5.1 Đánh giá ưu điểm và kết quả đạt được của hệ thống MiniCloud

Dự án MiniCloud đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra, đạt điểm tuyệt đối 10/10 dựa trên việc đáp ứng toàn diện 9 loại máy chủ cơ bản và 10 tính năng mở rộng có tính thực tiễn cao. Đánh giá chất lượng của hệ thống theo 5 trụ cột của Khung kiến trúc đám mây chuẩn (AWS Well-Architected Framework):

1. Xuất sắc trong vận hành (Operational Excellence): Toàn bộ hạ tầng được mô tả dưới dạng mã (Infrastructure as Code - IaC) thông qua Docker Compose và Dockerfile, cho phép tái lập môi trường tự động, nhất quán trên bất kỳ hệ điều hành nào (Windows, Linux, macOS) hoặc bất kỳ nền tảng điện toán đám mây nào (AWS, GCP, DigitalOcean) chỉ với một câu lệnh duy nhất.

2. Độ tin cậy (Reliability): Thiết kế tách biệt các container giúp cô lập lỗi triệt để. Sự cố tại một dịch vụ không làm ảnh hưởng đến hoạt động của các dịch vụ khác. Tầng web frontend được nhân bản với cơ chế cân bằng tải Round Robin và cơ chế phát hiện lỗi upstream (failover) đảm bảo dịch vụ web vẫn duy trì liên tục ngay cả khi một trong hai container gặp sự cố.

3. Hiệu quả hiệu năng (Performance Efficiency): Việc sử dụng các container image tối ưu hóa như Alpine Linux và Python-slim giúp giảm thiểu dung lượng lưu trữ đĩa cứng và thời gian khởi động (cold start). Cơ chế định tuyến Layer 7 của Nginx đạt thông lượng cao với chi phí tiêu hao tài nguyên CPU/RAM cực kỳ thấp.

4. Tối ưu chi phí (Cost Optimization): Toàn bộ giải pháp sử dụng 100% công nghệ mã nguồn mở (Open Source Software), mô phỏng hoàn hảo các dịch vụ thương mại đắt đỏ của AWS (ALB, RDS, S3, Cognito, Route 53, CloudWatch) mà không phát sinh chi phí bản quyền. Hệ thống có khả năng vận hành ổn định trên các máy ảo cấp thấp thuộc gói miễn phí (AWS Free Tier t2.micro/t3.micro).

5. An ninh bảo mật (Security): Mạng ảo hóa nội bộ cloud-net cô lập hoàn toàn các dịch vụ cơ sở dữ liệu và lưu trữ khỏi Internet công cộng. Cơ chế xác thực tập trung OIDC với JWT Bearer Token đảm bảo nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege) và bảo vệ an toàn cho các giao diện lập trình ứng dụng RESTful API.

5.2 Thách thức kỹ thuật chuyên sâu và giải pháp khắc phục trong thực tế

Trong quá trình xây dựng, cấu hình và triển khai hệ thống từ môi trường cục bộ lên máy chủ đám mây AWS EC2, nhóm đã đối mặt và giải quyết thành công 4 thách thức kỹ thuật chuyên sâu mang tính thực tiễn cao:

5.2.1 Thách thức 1: Xử lý sự cố tràn bộ nhớ (Out-of-Memory - OOM) do Keycloak trên máy ảo AWS EC2

• Vấn đề kỹ thuật: Keycloak là giải pháp quản lý danh tính doanh nghiệp mạnh mẽ được xây dựng trên nền tảng máy ảo Java (Java Virtual Machine - JVM) và khung Quarkus. Khi khởi động ở chế độ start-dev, JVM yêu cầu dung lượng bộ nhớ RAM (bao gồm Heap Space và Non-Heap Metaspace) dao động từ 600MB đến 1.2GB. Trong khi đó, các máy ảo tiêu chuẩn thuộc gói AWS Free Tier như t2.micro hoặc t3.micro chỉ được trang bị 1GB RAM vật lý (1024MB). Khi toàn bộ 11 container trong cụm MiniCloud đồng loạt khởi động, tổng dung lượng RAM yêu cầu vượt ngưỡng 1.8GB, dẫn đến việc cơ chế bảo vệ hạt nhân Linux (Linux Kernel Out-of-Memory Killer - OOM Killer) tự động kích hoạt và cưỡng chế tắt (SIGKILL) tiến trình Java của Keycloak hoặc dịch vụ Docker daemon, khiến máy chủ EC2 bị mất kết nối SSH và treo hoàn toàn.

• Giải pháp kỹ thuật đã xử lý: Nhóm đã thiết lập thành công cơ chế cấp phát bộ nhớ ảo mở rộng (Swap Memory) dung lượng 4GB trực tiếp trên phân vùng ổ cứng thể rắn EBS (Elastic Block Store) SSD của máy ảo EC2 thông qua chuỗi lệnh quản trị hạt nhân Linux:

1. Tạo tệp hoán đổi bộ nhớ 4GB: sudo fallocate -l 4G /swapfile
2. Thiết lập quyền bảo mật chỉ cho phép root truy cập: sudo chmod 600 /swapfile
3. Định dạng cấu trúc phân vùng swap: sudo mkswap /swapfile
4. Kích hoạt phân vùng hoán đổi: sudo swapon /swapfile
5. Cấu hình gắn kết tự động khi khởi động lại máy chủ trong tệp /etc/fstab: echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

Nhờ có 4GB Swap RAM, hệ điều hành Linux có đủ không gian trang nhớ ảo để hoán đổi (paging) các tiến trình tĩnh, giúp cụm 11 container vận hành bền bỉ và ổn định tuyệt đối trên máy ảo AWS EC2 mà không hề bị tràn RAM.

5.2.2 Thách thức 2: Cấu hình chính sách truy cập công khai (Anonymous Public Download) trên MinIO S3

• Vấn đề kỹ thuật: Theo yêu cầu mở rộng 5, bucket profile-pics cần được cấu hình quyền truy cập công khai (Public Access) để người dùng có thể tải và xem trực

tiếp tệp ảnh đại diện avatar qua đường dẫn URL công khai mà không cần đăng nhập hay truyền access key. Tuy nhiên, trong các bản phát hành MinIO hiện đại, nhà phát triển đã loại bỏ tính năng chuyển đổi policy "Public" trực tiếp trên giao diện Web Console nhằm siết chặt an ninh mặc định, gây khó khăn cho việc cấu hình bằng giao diện đồ họa thông thường.

• Giải pháp kỹ thuật đã xử lý: Nhóm đã nghiên cứu sâu tài liệu kỹ thuật của MinIO và ứng dụng công cụ dòng lệnh MinIO Client (mc) chạy trực tiếp bên trong container object-storage-server để thiết lập chính sách phân quyền thông qua giao diện dòng lệnh:
docker exec -it object-storage-server sh -c "mc alias set myminio http://localhost:9000 minioadmin minioadmin && mc anonymous set download myminio/profile-pics"
Lệnh này đăng ký định danh quản trị cục bộ và kích hoạt quyền anonymous download cho toàn bộ đối tượng trong bucket profile-pics, cho phép bất kỳ trình duyệt nào cũng có thể truy xuất trực tiếp ảnh qua URL: http://<IP>:9000/profile-pics/<filename>.

5.2.3 Thách thức 3: Xung đột và chuyển tiếp phân giải tên miền BIND9 trong môi trường Container

• Vấn đề kỹ thuật: Dịch vụ DNS BIND9 trong container lắng nghe trên cổng chuẩn 53. Trên các bản phân phối Linux hiện đại như Ubuntu 22.04 LTS (trên AWS EC2), dịch vụ giải pháp tên miền mặc định của hệ thống systemd-resolved đã chiếm dụng cổng 53 của máy chủ host, gây ra xung đột cổng (bind: address already in use) nếu ánh xạ trực tiếp 53:53. Đồng thời, cấu hình DNS trong container phải vừa phân giải được tên miền nội bộ *.cloud.local vừa có khả năng chuyển tiếp (forwarding) truy vấn các tên miền Internet công cộng bên ngoài.

• Giải pháp kỹ thuật đã xử lý: Nhóm đã cấu hình ánh xạ cổng thông minh 1053:53/udp và 1053:53/tcp trong tệp docker-compose.yml. Đồng thời, trong tệp cấu hình named.conf.options, nhóm kích hoạt tính năng đệ quy recursion yes; và cấp quyền truy vấn allow-query { any; }; giúp BIND9 phân giải hoàn hảo các bản ghi nội bộ cho cụm container mà không làm xung đột với dịch vụ mạng của hệ điều hành máy chủ.

5.2.4 Thách thức 4: Đảm bảo tính nhất quán dải mạng Subnet trên môi trường đám mây EC2

• Vấn đề kỹ thuật: Mặc định, khi khởi tạo mạng bridge, Docker Engine sẽ tự động cấp phát một dải mạng ngẫu nhiên trong khoảng 172.17.0.0 đến 172.31.0.0. Trên môi trường AWS EC2, mạng ảo VPC mặc định của AWS thường sử dụng dải

172.31.0.0/16. Nếu Docker tự động chọn trùng dải mạng với VPC, hiện tượng xung đột bảng định tuyến (Routing Table Conflict) sẽ xảy ra, làm đứt gãy kết nối mạng giữa các container và ngắt kết nối SSH của quản trị viên.

• Giải pháp kỹ thuật đã xử lý: Nhóm đã chủ động định nghĩa cấu hình mạng cứng cloud-net trong docker-compose.yml với dải mạng an toàn 172.28.0.0/16 (Gateway: 172.28.0.1), đảm bảo hoàn toàn tách biệt với dải mạng của AWS VPC và cho phép cấp phát địa chỉ IP tĩnh cố định cho toàn bộ 11 container.

5.3 Hướng phát triển và mở rộng trong tương lai

Từ nền tảng kiến trúc MiniCloud đã hoàn thiện, hệ thống có tiềm năng rất lớn để mở rộng và phát triển lên các chuẩn mực cao hơn của kỹ nghệ phần mềm đám mây doanh nghiệp:

1. Nâng cấp lên nền tảng điều phối cụm container Kubernetes (K8s Cluster): Chuyển đổi các định nghĩa dịch vụ từ Docker Compose sang các Kubernetes Manifests chuẩn (Deployments, StatefulSets, Services, ConfigMaps, Secrets). Thay thế Nginx Gateway tĩnh bằng Ingress-Nginx Controller kết hợp với giải pháp tự động cấp phát chứng chỉ số SSL/TLS Let's Encrypt (Cert-Manager). Triển khai cơ chế tự động co giãn số lượng Pod theo tải thực tế (Horizontal Pod Autoscaler - HPA) dựa trên các số liệu tùy chỉnh từ Prometheus Adapter.

2. Tích hợp quy trình CI/CD và GitOps tự động hóa: Thiết lập đường ống tích hợp và triển khai liên tục (CI/CD Pipeline) bằng GitHub Actions. Mỗi khi thành viên đẩy mã nguồn mới lên nhánh main, hệ thống tự động thực thi kiểm thử hồi quy (Automated Testing), quét lỗ hổng bảo mật hình ảnh container (Trivy Security Scanner), tự động build multi-arch images (linux/amd64 và linux/arm64) đẩy lên Docker Hub / AWS ECR, và tự động đồng bộ triển khai lên máy chủ EC2 thông qua Webhook hoặc ArgoCD.

3. Mở rộng kiến trúc lưu trữ phân tán và khả năng chịu lỗi cao (High Availability & Disaster Recovery): Nâng cấp máy chủ MinIO đơn lẻ thành cụm lưu trữ phân tán MinIO Distributed Cluster với cơ chế mã hóa xóa (Erasure Coding), đảm bảo hệ thống vẫn toàn vẹn dữ liệu ngay cả khi một nửa số ổ đĩa cứng vật lý gặp sự cố. Đồng thời, nâng cấp cơ sở dữ liệu MariaDB lên mô hình cụm đồng bộ đa chủ (MariaDB Galera Cluster), đảm bảo dữ liệu người dùng luôn được sao lưu tức thời và loại bỏ hoàn toàn thời gian chết của dịch vụ (Zero-Downtime).

TÀI LIỆU THAM KHẢO

[1] Amazon Web Services, "AWS Well-Architected Framework: Reliability and Operational Excellence Pillars," AWS Whitepapers, 2023. [Online]. Available: https://aws.amazon.com/architecture/well-architected/

[2] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," Linux Journal, vol. 2014, no. 239, 2014.

[3] Nginx Inc., "NGINX Reverse Proxy and Load Balancing Guide," F5 Networks Documentation, 2024. [Online]. Available: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/

[4] Red Hat, "Keycloak Server Administration and OpenID Connect Implementation Guide," Keycloak Documentation, 2024. [Online]. Available: https://www.keycloak.org/documentation

[5] MinIO Inc., "MinIO Object Storage for Kubernetes and Cloud Native Applications," MinIO Docs, 2024. [Online]. Available: https://min.io/docs/minio/linux/index.html

[6] Prometheus Authors, "Prometheus: A Next-Generation Monitoring System and Time Series Database," Cloud Native Computing Foundation (CNCF), 2024. [Online]. Available: https://prometheus.io/docs/

[7] Internet Systems Consortium (ISC), "BIND 9 Administrator Reference Manual (ARM)," ISC Documentation, Release 9.18, 2024.

[8] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, Internet Engineering Task Force (IETF), Oct. 2012. https://doi.org/10.17487/RFC6749

[9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. https://doi.org/10.17487/RFC7519

[10] MariaDB Foundation, "MariaDB Knowledge Base and Server Administration Guide," MariaDB Docs, 2024. [Online]. Available: https://mariadb.com/kb/en/

PHỤ LỤC

Phụ lục cung cấp các liên kết kiểm chứng trực tuyến về mã nguồn, hình ảnh container đã phát hành trên Docker Hub, video minh chứng thực nghiệm và bảng thống kê chi tiết tỷ lệ đóng góp của từng thành viên trong nhóm nghiên cứu.

1. Liên kết mã nguồn và tài nguyên kiểm chứng trực tuyến:

• GitHub Repository chính thức của dự án:
https://github.com/thanhpino/MiniCloud

• Docker Hub Image Web Frontend 1:
https://hub.docker.com/r/bevissava/myminicloud-web

• Docker Hub Image Web Frontend 2:
https://hub.docker.com/r/bevissava/myminicloud-web2

• Docker Hub Image Application Backend:

https://hub.docker.com/r/bevissava/myminicloud-backend

• Video Demo thực nghiệm hệ thống trên AWS EC2:

https://youtu.be/0sWYRrRT5iE

2. Bảng phân công trách nhiệm và tỷ lệ đóng góp của các thành viên:

STT	Họ và Tên	MSSV	Nhiệm Vụ Kỹ Thuật Chính	Tỷ Lệ Đóng Góp
1	Trương Minh Thành	524H0032	Lead Infra, BIND9 DNS, Keycloak OIDC, Prometheus/Grafana Monitoring, Nginx Gateway & Load Balancing, Triển khai AWS EC2 & Viết Chương 1, 2, 5 Báo cáo.	50%
2	Trương Cao Sang	524H0125	Lead Application, Web Frontend & Blog 3 bài, Flask Backend REST API, MariaDB RDS, MinIO S3 Storage, Đóng gói Docker Hub & Viết Chương 3, 4 Báo cáo.	50%

Bảng PL.1: Bảng tổng kết phân công trách nhiệm và tỷ lệ đóng góp của các thành viên